



## **CICLO FORMATIVO DE GRADO SUPERIOR EN DAM/DAW**

---

### **PROYECTO FIN DE CICLO**

Título: Vamin HUB

Alumno:

N. Alumno:

## DECLARACIÓN JURADA DE AUTORÍA

D. / Dña. Nombre\_alumno, con N.I.F. num\_DNI, como autor/es de este documento académico, titulado:

Vamin HUB

y presentado como Proyecto Fin de Ciclo para la obtención del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web,

DECLARO QUE

Soy/somos el/los único/s autor/es del trabajo, con la excepción de referencias a contenidos o ideas de otros autores, en cuyo caso han sido explícitamente citados.

El trabajo remitido es un documento original, que no ha sido publicado, ya sea total o parcialmente, ni presentado para obtención de un título académico en ninguna institución académica u organización.

No he trasgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual.

Soy consciente de que el hecho de no respetar estos extremos es una falta grave de integridad académica y podrá ser objeto de sanciones.

En Madrid, a fecha de entrega

Fdo.

## RESUMEN

Vamin HUB es un ERP desarrollado para una empresa fabricante de ventanas, optimizando sus procesos de fabricación de ventanas. El sistema centraliza la gestión de clientes, mediciones, obras, incidencias, montajes, producción, administración y contabilidad. Utiliza PHP para el backend y React para el frontend, con una base de datos en MySQL. Desplegado en servidores Amazon EC2, Vamin HUB mejora la eficiencia operativa integrando completamente los datos y procesos de la empresa.

## ABSTRACT

Vamin HUB is an ERP developed for a windows manufacturer, optimizing its window manufacturing processes. The system centralizes customer management, measurements, projects, incidents, installations, production, administration, and accounting. It uses PHP for the backend and React for the frontend, with a MySQL database. Deployed on Amazon EC2 servers, Vamin HUB enhances operational efficiency by fully integrating the company's data and processes.

## Contenido

|                                           |    |
|-------------------------------------------|----|
| 1. INTRODUCCIÓN                           | 4  |
| 1.1. CONTEXTO DEL PROYECTO                | 4  |
| 1.2. ELECCIÓN DEL TEMA                    | 4  |
| 1.3. OBJETIVO DEL PROYECTO                | 4  |
| 2. REQUISITOS FUNCIONALES                 | 5  |
| 2.1. INICIO                               | 5  |
| 2.2. CRM                                  | 5  |
| 2.3. PRODUCCIÓN                           | 6  |
| 2.4. ADMINISTRACIÓN                       | 6  |
| 2.5. CONTABILIDAD                         | 6  |
| 3. DISEÑO DE LA SOLUCIÓN                  | 7  |
| 3.1. Mapa de navegación                   | 7  |
| 3.2. Modelo de base de datos              | 7  |
| 3.3. Diseño de la arquitectura            | 10 |
| 3.4. Lenguajes utilizados                 | 12 |
| 4. DISEÑO TÉCNICO                         | 13 |
| 4.1. Libro de estilo                      | 13 |
| 5.2. Definición de componentes            | 17 |
| 5.3. Diseño de tablas de la base de datos | 49 |
| 6. MANUAL DE USUARIO                      | 50 |
| 7. DEMOSTRACIÓN                           | 54 |
| 7.2. Despliegue del Front-End en React    | 54 |
| 7.3. Despliegue del Back-End en PHP       | 54 |
| 7.4. Sistema de Despliegue Continuo       | 55 |
| 8. BIBLIOGRAFÍA                           | 56 |

## 1. INTRODUCCIÓN

### 1.1. CONTEXTO DEL PROYECTO

Este proyecto surge como parte de mi formación en el grado superior de Desarrollo de Aplicaciones Web (DAW). Consiste en el desarrollo de un sistema ERP personalizado para Ventanas Vamin, nuestro negocio familiar dedicado a la fabricación de ventanas. El objetivo es digitalizar y optimizar los procesos internos del negocio utilizando las habilidades y conocimientos adquiridos durante mi formación.

### 1.2. ELECCIÓN DEL TEMA

La decisión de centrar mi proyecto final en el desarrollo de un ERP personalizado para Ventanas Vamin surge de la evolución y las necesidades actuales de nuestro negocio familiar. Hasta ahora, he implementado una solución de software utilizando herramientas no code que ha resuelto varios problemas operativos. Sin embargo, esta solución presenta limitaciones, especialmente en términos de usabilidad y flexibilidad, que dificultan su adaptación a las crecientes demandas del negocio. En la actualidad, utilizamos una variedad de sistemas y métodos para gestionar diferentes aspectos del negocio. Esta situación ha resultado en un entorno donde los datos y procesos no están completamente integrados, lo que puede ser un desafío para mantener una gestión cohesiva y eficiente a medida que la empresa crece. Aunque Ventanas Vamin ha experimentado un crecimiento significativo en los últimos años, este éxito trae consigo la necesidad de una mayor digitalización. El objetivo no es simplemente reemplazar los sistemas existentes, sino crear una solución más robusta y escalable que pueda soportar y fomentar nuestro continuo crecimiento. Un ERP hecho a medida y desde cero, utilizando las tecnologías que he estudiado, nos permitiría unificar nuestros procesos, mejorar la eficiencia operativa y manejar mejor el nivel de demanda que estamos experimentando.

### 1.3. OBJETIVO DEL PROYECTO

El propósito principal de este trabajo es desarrollar un ERP que se adapte específicamente a las necesidades de Ventanas Vamin, facilitando la gestión integral y la eficiencia operativa. Busco crear una herramienta robusta que no solo cumpla con los requisitos académicos de mi grado, sino que también sea fundamental para el futuro crecimiento y éxito del negocio familiar. El nombre elegido para este sistema es "Vamin HUB". Este nombre encapsula la esencia del sistema: actuar como un núcleo central ("HUB") donde se centralizan todas las operaciones y procesos del negocio.

## 2. REQUISITOS FUNCIONALES

### 2.1. INICIO

Los usuarios deben poder iniciar sesión utilizando credenciales proporcionadas.

### 2.2. CRM

- Mediciones:
  - Los usuarios deben poder crear solicitudes de medición.
  - Deben poder visualizar y modificar mediciones pendientes de citar.
  - Un calendario debe mostrar las mediciones, permitiendo su modificación al seleccionarl
  - Deben poder visualizar y modificar mediciones pendientes de presupuestar.
  - Deben poder visualizar y modificar mediciones finalizadas.
  - El sistema debe permitir la subida y descarga de un PDF relacionado con la medición.
  - Los técnicos deben poder asignar y programar citas para mediciones.
  - Las mediciones deben mostrarse en el calendario según su cita.
  - Las mediciones deben poder asociarse a una obra, que puede tener varias mediciones.
- Clientes:
  - Los usuarios deben poder visualizar, añadir, modificar y eliminar clientes con la posibilidad de buscarlos a través de un campo de texto.
  - Debe existir la posibilidad de ver las obras asociadas a cada cliente en sus detalles.
- Obras:
  - Los usuarios deben poder visualizar un listado de obras, clasificadas por material, y gestionarlas.
  - Deben poder acceder a los detalles de la obra.
- Incidencias:
  - Los usuarios deben poder generar y gestionar incidencias, modificándolas o eliminándolas según su estado de cita o finalización.
  - El sistema debe relacionar cada incidencia con una obra específica.
- Montajes:
  - Debe existir un calendario para visualizar y gestionar montajes, con la capacidad de crear, modificar o eliminarlos.

- Los montajes deben poder asignarse a equipos de montadores y a un vehículo específico.
- El sistema debe permitir la subida y descarga de un PDF relacionado con el montaje.

### 2.3. PRODUCCIÓN

- Los usuarios deben poder ver y gestionar fabricaciones por material, creando y modificando registros.
- Deben poder crear y administrar pedidos de materiales necesarios para la producción.

### 2.4. ADMINISTRACIÓN

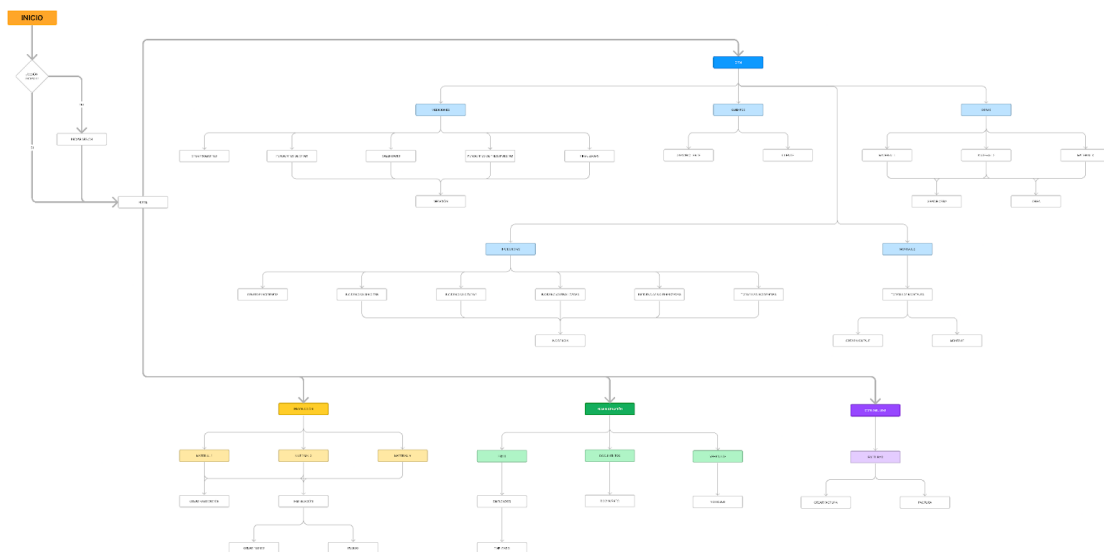
- Recursos Humanos (RRHH):
  - Debe ser posible la administración de empleados, incluyendo la creación, modificación y asignación de personal.
- Vehículos:
  - Los usuarios deben poder administrar la flota de vehículos, gestionar mantenimientos.
- Documentos:
  - El sistema debe permitir la gestión de documentos con fechas de vencimiento y la capacidad de cargar archivos

### 2.5. CONTABILIDAD

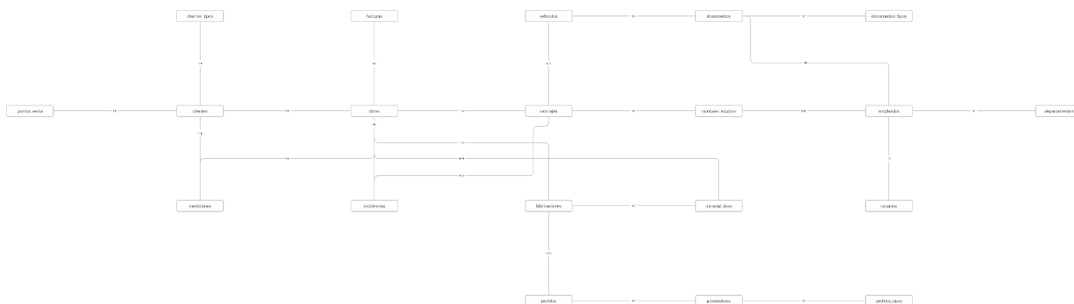
- Facturas:
  - Los usuarios deben poder crear, listar, filtrar y gestionar facturas asociadas a obras.

### 3. DISEÑO DE LA SOLUCIÓN

#### 3.1. Mapa de navegación



#### 3.2. Modelo de base de datos



- **Cientes**
  - Descripción: Almacena información sobre las personas o empresas que solicitan productos o servicios.
  - Atributos: ID del cliente, nombre, dirección, teléfono, email, tipo de cliente, notas, datos de facturación, punto de venta.
- **Puntos de Venta**
  - Descripción: Representa las ubicaciones físicas o virtuales donde los clientes realizan transacciones.
  - Atributos: ID del punto de venta, nombre, dirección.



- Clientes Tipos
  - Descripción: Clasifica a los clientes en categorías como particular, reformista, o distribuidor.
  - Atributos: ID del tipo de cliente, descripción del tipo.
- Obras
  - Descripción: Proyectos de fabricación e instalación de ventanas.
  - Atributos: ID de obra, número de presupuesto, referencia, cliente, fecha aceptado, fecha prevista, PDF del presupuesto, notas, días de montaje aprox, estado de la obra.
- Mediciones
  - Descripción: Detalles de las mediciones realizadas para las obras.
  - Atributos: ID de la medición, obra asociada, fecha solicitud, fecha cita, estado de la medición, pdf plantilla, pdf medido, nota.
- Facturas
  - Descripción: Documentos contables que certifican una transacción de venta.
  - Atributos: ID de la factura, obra, numero de factura, estado del pago.
- Incidencias
  - Descripción: Problemas o anomalías surgidas durante la ejecución de las obras o postventa.
  - Atributos: ID de la incidencia, descripción, obra asociada, fecha de reporte, estado de la incidencia.
- Montajes
  - Descripción: Información sobre la instalación de los productos en la obra.
  - Atributos: ID del montaje, obra asociada, equipo de montaje, fecha de montaje, estado del montaje, vehículo, pdf, notas.
- Vehículos
  - Descripción: Información sobre los vehículos utilizados para transportar materiales o equipos.
  - Atributos: Matrícula, modelo.
- Empleados
  - Descripción: Personal que trabaja en la empresa.
  - Atributos: ID del empleado, nombre, departamento, usuario del sistema.

- Documentos
  - Descripción: Gestión de documentos importantes para empleados y vehículos.
  - Atributos: ID del documento, tipo, empleado asociado, vehículo asociado, fecha de vencimiento, archivo adjunto.
- Documentos Tipos
  - Descripción: Categorías de los diferentes documentos gestionados.
  - Atributos: ID del tipo de documento, descripción.
- Montajes Equipos
  - Descripción: Grupos de empleados dedicados a la instalación en obra.
  - Atributos: ID del equipo, nombre.
- Departamentos
  - Descripción: Divisiones funcionales dentro de la empresa.
  - Atributos: ID del departamento, nombre.
- Usuarios
  - Descripción: Cuentas de acceso al sistema para empleados.
  - Atributos: ID de usuario, user, email, contraseña, rol usuario.
- Roles de usuario
  - Descripción: Roles para categorizar usuarios y en un futuro administrar los permisos.
  - Atributos: ID de rol, rol.
- Fabricaciones
  - Descripción: Registra la producción de las ventanas.
  - Atributos: ID de fabricación, obra asociada, estado de la fabricación, tipo de servicio, lote, fecha de fabricación, nota.
- Material Tipos
  - Descripción: Clasificación de los materiales utilizados en fabricaciones y pedidos
  - Atributos: ID del material, nombre.
- Pedidos
  - Descripción: Órdenes de compra de materiales para las fabricaciones.
  - Atributos: ID del pedido, proveedor, fecha de pedido, fecha de entrega, estado del pedido, pdf, notas.

- Pedidos Tipos
  - Descripción: Categoriza los pedidos para gestionar y diferenciar las diversas clases de órdenes de compra, como materia prima, accesorios, etc.
  - Atributos: ID del tipo de pedido, descripción del tipo.
- Proveedores
  - Descripción: Empresas o individuos que suministran materiales o servicios.
  - Atributos: ID del proveedor, nombre, tipo de pedido.

### 3.3. Diseño de la arquitectura

#### Back-End:

Framework Personalizado (PHP):

- Este framework personalizado, inspirado en Laravel, se encarga de la lógica de negocio, incluyendo el proceso de inicio de sesión, registro de usuarios, gestión de clientes, obras, mediciones, fabricaciones, incidencias, montajes, administración, contabilidad y análisis.
- Proporciona una API REST para la comunicación con el front-end, manejo de sesiones, y autenticación de usuarios, similar a cómo lo haría Laravel, pero con características y optimizaciones específicas al proyecto.

#### Front-End:

React:

- Se desarrolla una aplicación SPA (Single Page Application) utilizando React, lo cual permite una experiencia de usuario fluida y dinámica.
- React se encarga de consumir la API REST desarrollada por el backend, facilitando la interacción con los datos de manera asíncrona y mejorando la responsividad de la aplicación.
- La aplicación React incluye componentes reutilizables que mejoran la eficiencia del desarrollo y ofrecen una interfaz de usuario consistente y atractiva.

### **Servidor API:**

Nginx con PHP en Amazon EC2:

- Este servidor será responsable de manejar las solicitudes de la parte back-end utilizando PHP.
- Residirá en una instancia de Amazon EC2, ofreciendo escalabilidad y seguridad.
- Se configurará para utilizar HTTPS, asegurando que todas las comunicaciones entre el cliente y el servidor sean seguras.

### **Servidor WEB:**

Nginx con Node en Amazon EC2:

- Este servidor será responsable de manejar las solicitudes de la parte front de la aplicación .
- El servidor compila el código jsx de react para que nginx pueda servirlo.
- Residirá en una instancia de Amazon EC2, ofreciendo escalabilidad y seguridad.
- Se configurará para utilizar HTTPS, asegurando que todas las comunicaciones entre el cliente y el servidor sean seguras.

### **Gestor de Base de Datos:**

MySQL:

- Almacenará todas las entidades y relaciones descritas.
- La conexión se abrirá desde el framework PHP, utilizando un ORM para las operaciones de base de datos.
- Se elegirá MySQL por su compatibilidad con PHP y su rendimiento en operaciones transaccionales.
- Residirá en el servidor de la API, realizando las peticiones en la misma máquina.

### **Desarrollo e integración continuo:**

GitHub Actions:

- Con el servicio de las github actions se implementará el desarrollo e integración continuo en el que gracias a la gestión del proyecto de git se puede automatizar la actualización del código en los servidores de producción.
- Como la aplicación consta de dos proyectos git, funcionará con dos actions independientes para la actualización y realización de tareas necesarias para implementar las nuevas funciones en producción.

### 3.4. Lenguajes utilizados

#### **Framework Personalizado (PHP):**

- Aunque inspirado en Laravel, el uso de PHP se mantiene para desarrollar la lógica del servidor, manejar las solicitudes y respuestas HTTP, e interactuar con la base de datos MySQL. El enfoque está en ejecutar el backend de la aplicación, incluyendo autenticación de usuarios y operaciones CRUD.

#### **React:**

- React es el lenguaje principal para el frontend, permitiendo la creación de interfaces de usuario dinámicas que interactúan con el backend a través de la API REST. Se utilizan características modernas de JavaScript para mejorar la interactividad y la experiencia del usuario.

#### **SQL:**

- Para la manipulación y recuperación de datos en la base de datos MySQL, ejecutando consultas, actualizaciones, inserciones y eliminaciones de registros.

## 4. DISEÑO TÉCNICO

### 4.1. Libro de estilo



#### ABOUT

Vamin HUB representa una innovación clave para Ventanas Vamin, ofreciendo una solución especializada en sistemas ERP y CRM. Diseñado específicamente para mejorar la eficiencia y efectividad en la gestión de procesos en nuestra fábrica de ventanas, Vamin HUB integra tecnologías avanzadas con prácticas de desarrollo ágil. Con un enfoque en la escalabilidad y seguridad, proporcionado por plataformas como Amazon EC2, y la excelencia en codificación PHP con Laravel, este proyecto es el núcleo de nuestra transformación digital, llevando a Ventanas Vamin hacia un futuro tecnológicamente avanzado y operativamente superior.

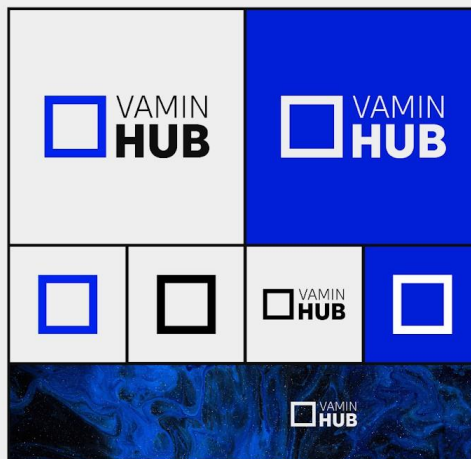
#### FONT

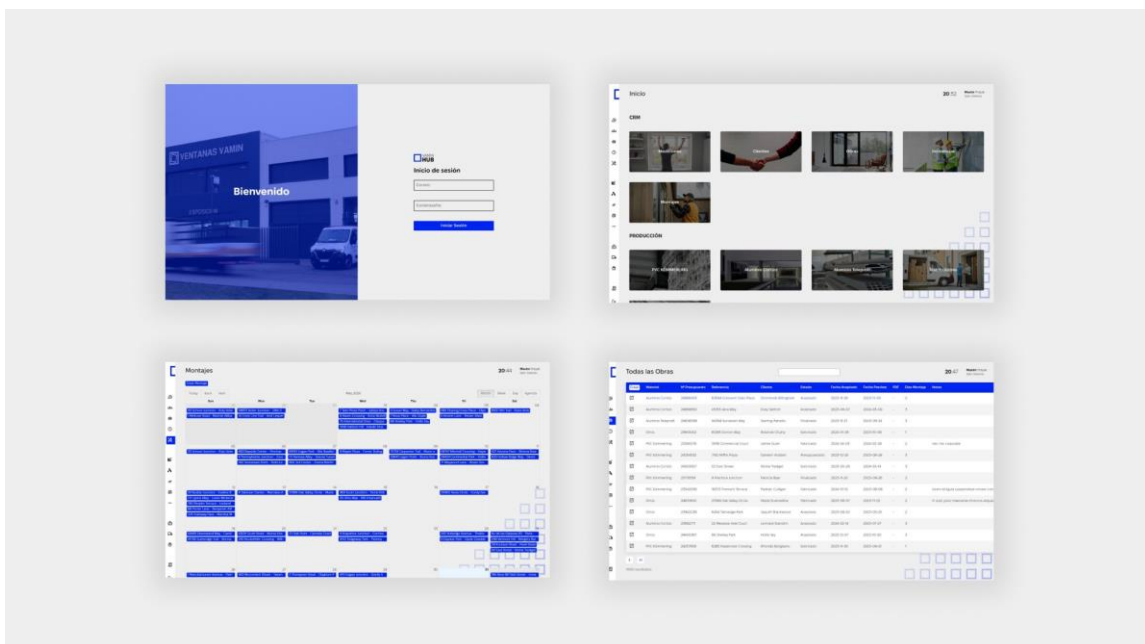
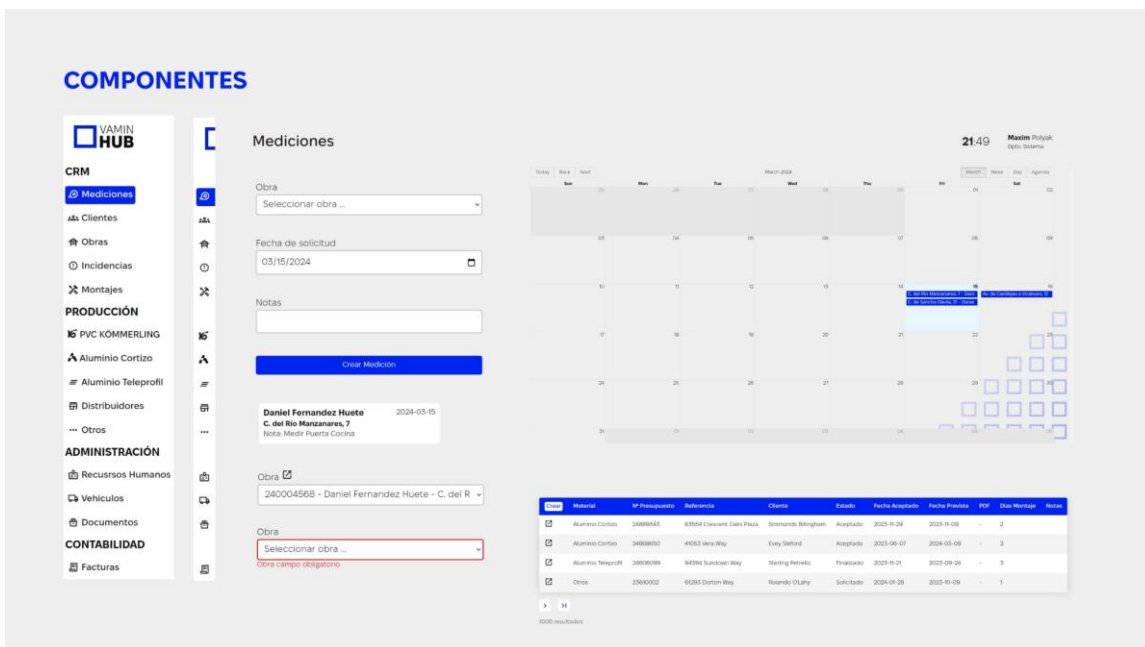
Fixel is a typeface in two styles, Text and Display, and offers nine weight options ranging from **Thin** to **Black**.

#### COLOR PALLETE

#0024EC  #EEEEEE  #0F0F0F

#### LOGOS





## Rejilla Utilizada

La rejilla utilizada en Vamin HUB sigue una estructura flexible adaptada a los diferentes tamaños de pantalla, utilizando principalmente un sistema de rejilla basado en CSS Grid y Flexbox para asegurar una disposición responsiva y ordenada de los elementos.

## Logotipo

Diseño: El logotipo debe mantener la identidad visual de Ventanas Vamin, con un estilo moderno y claro.

Colocación: El logotipo se coloca en la esquina superior izquierda de todas las páginas principales y en el encabezado del menú lateral.

Colores: El logotipo utiliza los colores principales de la paleta (#0024EC y #0F0F0F).

Tamaño: El logotipo debe mantenerse en proporciones que aseguren su legibilidad y presencia, usualmente no menor a 50px de alto.

## Tipografía

Fuente Principal: Fixel

Estilos y Pesos:

- Regular (400)
- Light (300)
- Bold (700)
- Medium (500)

Uso:

- Encabezados: Fixel Bold, tamaño entre 20px y 30px, dependiendo del nivel del encabezado.
- Texto Normal: Fixel Regular, tamaño 16px.
- Etiquetas y Inputs: Fixel Light, tamaño 14px a 20px.
- Altura de Línea: 1.5 para asegurar legibilidad.
- Espaciado entre Letras: Normal (letter-spacing: normal).

## Paleta de Colores

Colores Primarios:

- Azul Principal: #0024EF
- Contraste: #FFFFFF

Colores Secundarios:

- Fondo Oscuro: #1E1E1E
- Fondo Claro: #FFFFFF



Colores de Acento:

- Error: #e10000
- Éxito: #09c403

### **Imágenes y Gráficos**

Calidad: Alta calidad, preferiblemente en formato WebP.

Tamaño: Adaptativo, pero asegurando una resolución mínima de 72 DPI.

Gráficos y Iconos: Vectoriales (SVG) para asegurar escalabilidad sin pérdida de calidad.

### **Hojas de Estilo y su ubicación**

calendar.css: Estilos para el componente del calendario.

formulario.css: Estilos para los formularios de entrada.

init.css: Configuración inicial y fuentes.

log-in.css: Estilos para la página de inicio de sesión.

main.css: Estilos generales para la estructura principal.

pendientes.css: Estilos para la vista de mediciones pendientes.

presupuestar.css: Estilos para la vista de mediciones a presupuestar.

menu.css: Estilos para el menú de navegación.

side-bar.css: Estilos para la barra lateral.

table.css: Estilos para las tablas de datos.

theme-dark.css: Estilos para el tema oscuro.

theme-light.css: Estilos para el tema claro.

top-menu.css: Estilos para el menú superior.

## 5.2. Definición de componentes

### 5.2.1. COMPONENTES FRAMEWORK PHP

En el siguiente apartado se describirán los componentes principales que conforman el framework, componentes cuales hacen posibles un desarrollo de api muy sencilla y escalable:

- **App.php:** Es el núcleo del framework de la aplicación. Este archivo se encarga de inicializar y configurar todos los componentes necesarios para que la aplicación funcione correctamente. Administra la configuración de la aplicación, la conexión a la base de datos, las sesiones de usuario, y el enrutamiento de las solicitudes HTTP.
- **Auth/Auth.php:** maneja la autenticación de usuarios dentro del sistema. Proporciona métodos para obtener el usuario autenticado actualmente, verificar si el usuario es un invitado y registrar las rutas relacionadas con la autenticación (inicio de sesión y cierre de sesión).
  - Método `user()`: Devuelve el usuario autenticado actual utilizando el autenticador configurado.
  - Método `isGuest()`: Verifica si no hay usuario autenticado.
  - Método `routes()`: Define las rutas relacionadas con la autenticación, como el inicio y cierre de sesión.
- **Auth/Authenticatable.php:** extiende la clase `Model` y define métodos para manejar la autenticación de la entidad mediante la interfaz `Authenticator`. Este archivo incluye métodos para obtener el ID del usuario, iniciar sesión, cerrar sesión y verificar el estado de autenticación.
  - Método `id()`: Devuelve el identificador único del usuario, que puede ser un entero o una cadena, basado en la clave primaria del modelo.
  - Método `login()`: Inicia sesión para el usuario actual utilizando el autenticador configurado.
  - Método `logout()`: Cierra la sesión del usuario actual utilizando el autenticador configurado.
  - Método `isAuthenticated()`: Verifica si el usuario actual está autenticado utilizando el autenticador configurado.

- **Auth/Authenticators/SessionAuthenticator.php:** Componente que implementa la interfaz Authenticator para manejar la autenticación de usuarios mediante sesiones. Este archivo contiene métodos para iniciar sesión, cerrar sesión, verificar si un usuario está autenticado y resolver el usuario autenticado actual desde la sesión.
  - Método login(Authenticatable \$authenticatable): Este método inicia la sesión para el usuario dado. Utiliza el objeto session para almacenar el usuario autenticado bajo la clave \_auth.
  - Método logout(Authenticatable \$authenticatable): Cierra la sesión del usuario proporcionado, eliminándolo de la sesión.
  - Método isAuthenticated(Authenticatable \$authenticatable): Verifica si el usuario proporcionado está autenticado comparando el ID del usuario almacenado en la sesión con el ID del usuario proporcionado.
  - Método resolve(): Devuelve el usuario autenticado actual desde la sesión, o null si no hay ningún usuario autenticado.
  
- **Container/Container.php:** Es un componente que gestiona las instancias de clases y proporciona un contenedor de servicios para la inyección de dependencias. Este archivo maneja la creación y resolución de instancias singleton, asegurando que cada clase tenga una única instancia compartida en toda la aplicación.
  - Método singleton(string \$class, string|callable|null \$build = null): Este método crea y devuelve una instancia singleton de la clase especificada. Si la instancia ya existe, la devuelve; de lo contrario, la crea utilizando la clase proporcionada, una cadena que representa una clase o una función de construcción.
  - Método resolve(string \$class): Este método resuelve y devuelve la instancia de la clase especificada si ya ha sido creada. Si no ha sido creada, devuelve null.

- **Container/DependencyInjection.php:** Es un componente que maneja la inyección de dependencias en el framework. Este archivo utiliza la reflexión para resolver automáticamente los parámetros necesarios para los métodos y funciones, facilitando así la construcción y gestión de objetos complejos.
  - Método `resolveParameters(\Closure|array $callback, $routeParameters = [])`: Este método resuelve los parámetros necesarios para un método o función proporcionada. Utiliza la reflexión para analizar los tipos de parámetros requeridos y los resuelve, ya sea recuperando instancias de modelos, parámetros integrados de la ruta, o creando instancias de clases utilizando el contenedor de servicios.
- **Database/Drivers/PdoDriver.php:** Implementa la interfaz `DatabaseDriver` utilizando PHP Data Objects (PDO) para gestionar la conexión y operaciones con la base de datos. Este archivo se encarga de establecer la conexión, ejecutar consultas, y manejar el cierre de la conexión.
  - Método `connect(string $protocol, string $host, int $port, string $database, string $username, string $password)`: Este método establece una conexión a la base de datos utilizando PDO. Configura la conexión con los parámetros proporcionados (protocolo, host, puerto, base de datos, usuario y contraseña) y establece el modo de error en excepciones.
  - Método `lastInsertId()`: Este método devuelve el ID del último registro insertado en la base de datos a través de esta conexión PDO.
  - Método `close()`: Este método cierra la conexión a la base de datos estableciendo la propiedad PDO a null.
  - Método `statement(string $query, array $bind = [])`: mixed: Este método prepara y ejecuta una consulta SQL con los parámetros vinculados proporcionados. Devuelve los resultados de la consulta como un array asociativo.

- **Database/Model.php:** Es una clase abstracta que representa un modelo de base de datos en el framework. Esta clase proporciona una estructura base para interactuar con la base de datos, facilitando la realización de operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y la gestión de atributos del modelo.
  - Propiedades:
    - `protected ?string $table = null`: Nombre de la tabla en la base de datos.
    - `protected ?string $primaryKey = null`: Clave primaria de la tabla.
    - `protected array $joins = []`: Configuración de joins con otras tablas.
    - `protected array $customJoins = []`: Configuración de joins personalizados.
    - `protected array $hidden = []`: Atributos que deben ser ocultos al serializar el modelo.
    - `protected array $fillable = []`: Atributos que se pueden asignar masivamente.
    - `protected array $attributes = []`: Atributos del modelo.
    - `protected bool $insertTimestamps = true`: Indica si se deben insertar marcas de tiempo.
    - `private static ?DatabaseDriver $driver = null`: Controlador de base de datos.
  - Método `setDatabaseDriver(DatabaseDriver $driver)`: Establece el controlador de la base de datos que utilizará el modelo.
  - Método `__construct()`: Constructor de la clase que inicializa la tabla y la clave primaria si no están definidas explícitamente.
  - Método `__set($name, $value)`: Asigna un valor a un atributo del modelo.
  - Método `__get($name)`: Obtiene el valor de un atributo del modelo.
  - Método `__sleep()`: Prepara el modelo para la serialización, eliminando los atributos ocultos.
  - Método `setAttributes(array $attributes): static`: Asigna múltiples atributos al modelo.
  - Método `massAssign(array $attributes): static`: Asigna atributos masivamente si están en la lista de fillables.
  - Método `toArray(): array`: Convierte el modelo a un array, excluyendo los atributos ocultos.
  - Método `save(): static`: Guarda el modelo en la base de datos, insertando un nuevo registro.

- Método `update()`: static: Actualiza un registro existente en la base de datos.
  - Método `delete()`: static: Marca un registro como inactivo en la base de datos.
  - Método `create(array $attributes)`: static: Crea y guarda un nuevo registro en la base de datos.
  - Método `first()`: ?static: Obtiene el primer registro activo de la tabla.
  - Método `find(int|string $id)`: ?static: Encuentra un registro por su ID.
  - Método `findAllObjects(array $ids)`: ?array: Encuentra múltiples registros por sus IDs y devuelve los objetos.
  - Método `findAll(array $ids)`: ?array: Encuentra múltiples registros por sus IDs y devuelve los arrays.
  - Método `allObjects()`: array: Obtiene todos los registros activos como objetos.
  - Método `all()`: array: Obtiene todos los registros activos como arrays.
  - Método `allJoinedObjects()`: array: Obtiene todos los registros activos con joins como objetos.
  - Método `allJoined()`: array: Obtiene todos los registros activos con joins como arrays.
  - Método `where(string $column, mixed $value)`: array: Encuentra registros basados en una condición de columna.
  - Método `firstWhere(string $column, mixed $value)`: ?static: Encuentra el primer registro basado en una condición de columna.
  - Método `lastInsert()`: ?static: Encuentra el último registro insertado.
  - Método `getSqlJoinedQuery($model, string $where = "")`: string: Genera la consulta SQL con joins personalizados.
- 
- **Http/Controller.php:** Es una clase base para todos los controladores en el framework. Proporciona métodos para gestionar los middlewares que se aplican a las acciones del controlador, facilitando la implementación de lógica adicional antes o después de la ejecución de una acción.
    - Propiedad `protected array $middlewares = []`: Almacena una lista de middlewares que se aplican a las acciones del controlador.
    - Método `middlewares()`: array: Devuelve la lista de middlewares asociados con el controlador.
    - Método `setMiddlewares(array $middlewares)`: self: Establece los middlewares para el controlador. Recibe un array de nombres de clases de middlewares,

crea nuevas instancias de cada middleware y las asigna a la propiedad \$middlewares.

- **Http/Request.php:** Es una clase que representa una solicitud HTTP en el framework. Esta clase encapsula la información de la solicitud, como la URI, el método HTTP, los datos de la solicitud, las cabeceras, los parámetros de la ruta y los archivos adjuntos. Proporciona métodos para acceder y manipular estos datos.
  - Propiedades:
    - protected string \$uri: URI de la solicitud.
    - protected Route \$route: Ruta asociada con la solicitud.
    - protected HttpMethod \$method: Método HTTP de la solicitud.
    - protected array \$data: Datos del cuerpo de la solicitud.
    - protected array \$query: Parámetros de la consulta.
    - protected array \$headers = []: Cabeceras de la solicitud.
    - protected array \$files = []: Archivos adjuntos en la solicitud.
  - Método uri(): string: Devuelve la URI de la solicitud.
  - Método setUri(string \$uri): self: Establece la URI de la solicitud y devuelve la instancia actual.
  - Método route(): Route: Devuelve la ruta asociada con la solicitud.
  - Método setRoute(Route \$route): self: Establece la ruta asociada con la solicitud y devuelve la instancia actual.
  - Método method(): HttpMethod: Devuelve el método HTTP de la solicitud.
  - Método setMethod(HttpMethod \$method): self: Establece el método HTTP de la solicitud y devuelve la instancia actual.
  - Método headers(string \$key = null): array|string|null: Devuelve todas las cabeceras o una cabecera específica de la solicitud.
  - Método setHeaders(array \$headers): self: Establece las cabeceras de la solicitud y devuelve la instancia actual.
  - Método data(?string \$key = null): array|string|null: Devuelve todos los datos del cuerpo de la solicitud o un dato específico.
  - Método setPostData(array \$data): self: Establece los datos del cuerpo de la solicitud y devuelve la instancia actual.
  - Método query(?string \$key = null): array|string|null: Devuelve todos los parámetros de la consulta o un parámetro específico.

- Método `setQueryParameters(array $query): self`: Establece los parámetros de la consulta y devuelve la instancia actual.
  - Método `routeParameters(?string $key = null): array|string|null`: Devuelve todos los parámetros de la ruta o un parámetro específico.
  - Método `validate(array $rules, array $messages = [])`: array: Valida los datos de la solicitud según las reglas proporcionadas y devuelve los errores de validación si los hay.
  - Método `file(string $name): ?File`: Devuelve un archivo específico de la solicitud.
  - Método `setFiles(array $files): self`: Establece los archivos adjuntos en la solicitud y devuelve la instancia actual.
- 
- **Http/Response.php**: Es una clase que representa una respuesta HTTP en el framework. Esta clase encapsula la información de la respuesta, como el estado HTTP, las cabeceras, el contenido y proporciona métodos para manipular estos datos y generar diferentes tipos de respuestas (JSON, texto, redirección, archivos).
    - Propiedad `protected int $status = 200`: Código de estado HTTP de la respuesta, por defecto 200 (OK).
    - Propiedad `protected array $headers = []`: Array asociativo de cabeceras HTTP.
    - Propiedad `protected ?string $content = null`: Contenido de la respuesta, inicialmente null.
    - Método `status(): int`: Devuelve el código de estado HTTP de la respuesta.
    - Método `setStatus(int $status): self`: Establece el código de estado HTTP de la respuesta y devuelve la instancia actual.
    - Método `headers(string $key = null): array|string|null`: Devuelve todas las cabeceras o una cabecera específica de la respuesta.
    - Método `setHeader(string $header, string $value): self`: Establece una cabecera HTTP y devuelve la instancia actual.
    - Método `removeHeader(string $header)`: Elimina una cabecera HTTP de la respuesta.
    - Método `setContentType(string $value): self`: Establece el tipo de contenido de la respuesta y devuelve la instancia actual.
    - Método `content(): ?string`: Devuelve el contenido de la respuesta.



- Método `setContent(string $content)`: self: Establece el contenido de la respuesta y devuelve la instancia actual.
  - Método `prepare()`: Prepara la respuesta antes de enviarla, ajustando las cabeceras según el contenido.
  - Método `json(array $data)`: self: Crea una respuesta JSON con los datos proporcionados.
  - Método `text(string $text)`: self: Crea una respuesta de texto plano con el contenido proporcionado.
  - Método `redirect(string $uri)`: self: Crea una respuesta de redirección a la URI proporcionada.
  - Método `file(?string $filepath)`: self: Crea una respuesta que entrega un archivo desde el almacenamiento.
- 
- **Routing/Route.php**: Es una clase que representa una ruta en el framework. Esta clase maneja la URI, la acción a realizar, las expresiones regulares para el enrutamiento y los parámetros de la ruta. También gestiona los middlewares asociados a la ruta y proporciona métodos estáticos para definir rutas HTTP.
    - Propiedades:
      - `protected string $uri`: URI de la ruta.
      - `protected \Closure|array $action`: Acción asociada a la ruta, puede ser un Closure o un array con controlador y método.
      - `protected string $regex`: Expresión regular para hacer coincidir la URI.
      - `protected array $parameters`: Parámetros extraídos de la URI.
      - `protected array $middlewares = []`: Middlewares asociados a la ruta.
    - Constructor `__construct(string $uri, \Closure|array $action)`: Inicializa una nueva instancia de la clase Route con la URI y la acción asociada. Configura la expresión regular para la URI y extrae los parámetros de la URI.
    - Método `uri()`: string: Devuelve la URI de la ruta.
    - Método `action()`: \Closure|array: Devuelve la acción asociada a la ruta.
    - Método `middlewares()`: array: Devuelve los middlewares asociados a la ruta.
    - Método `setMiddlewares(array $middlewares)`: self: Establece los middlewares para la ruta y devuelve la instancia actual.
    - Método `hasMiddlewares()`: bool: Verifica si la ruta tiene middlewares.

- Método `matches(string $uri): bool`: Comprueba si la URI proporcionada coincide con la expresión regular de la ruta.
- Método `hasParameters(): bool`: Verifica si la ruta tiene parámetros.
- Método `parseParameters(string $uri): array`: Extrae y devuelve los parámetros de la URI proporcionada.
- Método estático `load(string $routesDirectory)`: Carga todos los archivos de rutas desde el directorio especificado.
- Método estático `get(string $uri, Closure|array $action): Route`: Define una ruta GET.
- Método estático `post(string $uri, Closure|array $action): Route`: Define una ruta POST.
- Método estático `put(string $uri, Closure|array $action): Route`: Define una ruta PUT.
- Método estático `delete(string $uri, Closure|array $action): Route`: Define una ruta DELETE.
- **Routing/Router.php**: Es una clase que maneja el enrutamiento de solicitudes HTTP en el framework. Esta clase registra las rutas, resuelve las rutas correspondientes a las solicitudes, ejecuta las acciones asociadas a las rutas y aplica los middlewares correspondientes.
  - Propiedad `protected array $routes = []`: Almacena las rutas registradas, organizadas por método HTTP.
  - Constructor `__construct()`: Inicializa el array de rutas para cada método HTTP disponible.
  - Método `resolveRoute(Request $request): Route`: Resuelve y devuelve la ruta correspondiente a la solicitud dada. Si no se encuentra ninguna ruta que coincida, lanza una excepción `HttpException`.
  - Método `resolve(Request $request): Response`: Resuelve la solicitud, ejecuta la acción asociada a la ruta y aplica los middlewares. Devuelve una respuesta HTTP.
  - Método `runMiddlewares(Request $request, array $middlewares, $target): Response`: Ejecuta los middlewares en cadena antes de ejecutar la acción principal de la ruta.
  - Método `registerRoute(HttpMethod $method, string $uri, Closure|array $action): Route`: Registra una nueva ruta con el método HTTP especificado, la URI y la acción asociada. Devuelve la instancia de la ruta registrada.

- Método `get(string $uri, Closure|array $action)`: Route: Registra una ruta GET.
  - Método `post(string $uri, Closure|array $action)`: Route: Registra una ruta POST.
  - Método `put(string $uri, Closure|array $action)`: Route: Registra una ruta PUT.
  - Método `patch(string $uri, Closure|array $action)`: Route: Registra una ruta PATCH.
  - Método `delete(string $uri, Closure|array $action)`: Route: Registra una ruta DELETE.
- 
- **Server/PhpNativeServer.php**: Es una clase que implementa la interfaz `Server` y gestiona la interacción con el servidor PHP nativo. Esta clase se encarga de construir la solicitud HTTP (`Request`) desde la superglobal `$_SERVER`, y de enviar la respuesta HTTP (`Response`) utilizando funciones nativas de PHP.
    - Método `getRequest()`: `Request`: Crea y devuelve una instancia de `Request` configurada con los datos de la solicitud HTTP actual.
    - Método `sendResponse(Response $response)`: Envía la respuesta HTTP al cliente, configurando las cabeceras y el contenido de la respuesta.
    - Método `uploadedFiles()`: `array`: Procesa los archivos subidos en la solicitud y devuelve un array de instancias de `File`.
    - Método `requestData()`: `array`: Procesa los datos del cuerpo de la solicitud y devuelve un array con los datos correspondientes.
- 
- **Session/PhpNativeSessionStorage.php**: Es una clase que implementa la interfaz `SessionStorage` y gestiona las sesiones utilizando las funciones nativas de PHP. Esta clase proporciona métodos para iniciar, guardar, manipular y destruir sesiones, facilitando el manejo de sesiones en la aplicación.
    - Método `start()`: Inicia una nueva sesión o reanuda una sesión existente. Si falla, lanza una excepción `\RuntimeException`.
    - Método `save()`: Escribe los datos de la sesión y cierra la sesión.
    - Método `id()`: `string`: Devuelve el ID de la sesión actual.
    - Método `get(string $key, $default = null)`: Recupera un valor de la sesión por su clave. Si la clave no existe, devuelve un valor por defecto.
    - Método `set(string $key, mixed $value)`: Establece un valor en la sesión para una clave dada.

- Método `has(string $key)`: bool: Verifica si una clave existe en la sesión.
  - Método `remove(string $key)`: Elimina un valor de la sesión por su clave.
  - Método `destroy()`: Destruye la sesión actual.
- 
- **Session/Session.php**: Es una clase que gestiona las sesiones en la aplicación utilizando una instancia de `SessionStorage`. Esta clase proporciona métodos para manipular los datos de la sesión, manejar datos flash y destruir la sesión.
    - Propiedad `protected SessionStorage $storage`: Almacena la instancia de `SessionStorage` utilizada para manejar la sesión.
    - Constante `public const FLASH_KEY = '_flash'`: Clave utilizada para almacenar los datos flash en la sesión.
    - Constructor `__construct(SessionStorage $storage)`: Inicializa la instancia de `Session` con el almacenamiento de sesiones proporcionado y empieza la sesión. Si los datos flash no existen, los inicializa.
    - Método `__destruct()`: Limpia los datos flash antiguos, actualiza los datos flash y guarda la sesión antes de destruir la instancia.
    - Método `ageFlashData()`: Actualiza los datos flash, moviendo los datos nuevos a la categoría de datos antiguos y limpiando los nuevos.
    - Método `flash(string $key, mixed $value)`: Almacena un dato flash en la sesión, que solo estará disponible en la siguiente solicitud.
    - Método `id()`: string: Devuelve el ID de la sesión actual.
    - Método `get(string $key, $default = null)`: Recupera un valor de la sesión por su clave. Si la clave no existe, devuelve un valor por defecto.
    - Método `set(string $key, mixed $value)`: Establece un valor en la sesión para una clave dada.
    - Método `has(string $key)`: bool: Verifica si una clave existe en la sesión.
    - Método `remove(string $key)`: Elimina un valor de la sesión por su clave.
    - Método `destroy()`: Destruye la sesión actual.

- **Storage/Drivers/DiskFileStorage.php:** Es una clase que implementa la interfaz `FileStorageDriver` para manejar el almacenamiento de archivos en el sistema de archivos local. Esta clase proporciona métodos para guardar y recuperar archivos desde el disco, facilitando la gestión de archivos en la aplicación.
  - Propiedades:
    - `protected string $storageDirectory`: Directorio base donde se almacenan los archivos.
    - `protected string $appUrl`: URL base de la aplicación.
    - `protected string $storageUri`: URI de almacenamiento.
  - Constructor `__construct(string $storageDirectory, string $storageUri, string $appUrl)`: Inicializa la instancia de `DiskFileStorage` con el directorio de almacenamiento, URI de almacenamiento y URL de la aplicación.
  - Método `put(string $path, mixed $content)`: `string`: Guarda un archivo en el sistema de archivos. Si el directorio de almacenamiento no existe, lo crea. También crea cualquier subdirectorio necesario y guarda el contenido en el archivo especificado.
  - Método `get(string $path)`: `?File`: Recupera un archivo del sistema de archivos. Si el archivo existe, crea y devuelve una instancia de `File` con el contenido, tipo MIME y nombre del archivo. Si el archivo no existe, devuelve `null`.
- **Storage/File.php:** Es una clase que representa un archivo dentro del sistema de almacenamiento del framework. Esta clase encapsula la información del archivo, como su contenido, tipo MIME y nombre original, y proporciona métodos para trabajar con archivos, incluyendo determinar si el archivo es una imagen, obtener su extensión, y almacenarlo en el sistema de archivos.
  - Propiedades:
    - `private mixed $content`: Contenido del archivo.
    - `private string $type`: Tipo MIME del archivo.
    - `private string $originalName`: Nombre original del archivo.
  - Constructor `__construct(mixed $content, string $type, string $originalName)`: Inicializa una instancia de `File` con el contenido, tipo y nombre original del archivo.
  - Método `isImage()`: `bool`: Determina si el archivo es una imagen comprobando si su tipo MIME empieza con "image".
  - Método `extension()`: `?string`: Devuelve la extensión del archivo basada en su tipo MIME. Si el tipo MIME no es reconocido, devuelve `null`.

- Método `store(?string $directory = null): string`: Almacena el archivo en el sistema de archivos. Genera un nombre único para el archivo con la extensión correspondiente y lo guarda en el directorio especificado (o en el directorio base si no se especifica ninguno).
- Método `getContent():` Devuelve el contenido del archivo.
- Método `getType():` Devuelve el tipo MIME del archivo.
- Método `getOriginalName():` Devuelve el nombre original del archivo.
- 
- **Validation/Rule.php**: Es una clase que gestiona las reglas de validación en el framework. Esta clase proporciona métodos para cargar y gestionar las reglas de validación, convertir las reglas a partir de cadenas de texto y crear instancias de reglas específicas.
  - Propiedades:
    - `private static array $rules = []`: Almacena las reglas de validación registradas.
    - `private static array $defaultRules = [ ]`: Lista de reglas de validación predeterminadas.
  - Método `loadDefaultRules():` Carga las reglas de validación predeterminadas.
  - Método `load(array $rules)`: Carga una lista de reglas de validación proporcionadas, registrando cada clase de regla con su nombre en formato `snake_case`.
  - Método `nameOf(ValidationRule $rule): string`: Devuelve el nombre de una regla de validación en formato `snake_case`.
  - Métodos de Creación de Reglas: Métodos estáticos que crean y devuelven instancias de reglas de validación específicas.
  - Método `parseBasicRule(string $ruleName): ValidationRule`: Convierte una cadena de texto a una regla básica de validación, sin parámetros.
  - Método `parseRuleWithParameters(string $ruleName, string $params): ValidationRule`: Convierte una cadena de texto a una regla de validación con parámetros.
  - Método `from(string $str): ValidationRule`: Convierte una cadena de texto en una regla de validación, manejando tanto reglas básicas como reglas con parámetros.

- **Validation/Validator.php:** Es una clase que se encarga de validar los datos de entrada en el framework. Esta clase utiliza las reglas de validación definidas para verificar los datos y lanza una excepción si alguna validación falla.
  - Propiedad protected array \$data: Almacena los datos que deben ser validados.
  - Constructor `__construct(array $data)`: Inicializa una nueva instancia de Validator con los datos a validar.
  - Método `validate(array $validationRules, array $messages = [])`: array: Valida los datos según las reglas de validación proporcionadas. Si alguna validación falla, lanza una excepción `ValidationException` con los errores de validación.

### 5.2.2. COMPONENTES API

En el siguiente apartado se describirán los archivos que conforman la api, una api desarrollada con el framework realizado, la estructura de archivos está hecha en MVC separando los controladores y los modelos en este caso, ya que no tenemos vistas, pero sí que tenemos archivos que crean las rutas a las cuales se llaman desde el front:

- **Models:** En el proyecto ERP, cada tabla de la base de datos tiene un modelo correspondiente en la API. Estos modelos extienden de una clase base Model, que proporciona funcionalidades ORM para interactuar con la base de datos. Los modelos tienen una estructura similar, con la capacidad de definir un array fillable para especificar los campos que pueden ser rellenados por el controlador. En algunos casos, se sobrescribe el constructor de la clase Model para especificar la tabla y la clave primaria de la base de datos, así como los joins con otras tablas.
  - Estructura General de los Modelos:
    - Nombre de la Tabla y Clave Primaria: En algunos modelos, se define explícitamente la tabla y la clave primaria en el constructor.
    - Campos Rellenables: Un array fillable que lista los campos que pueden ser rellenados.
    - Joins: En algunos modelos, se define un array joins para especificar cómo se deben unir otras tablas relacionadas.

○ Ejemplo Detallado: Modelo Cliente

```
<?php

namespace App\Models;

use api\Database\Model;

class Cliente extends Model{

    public function __construct() {
        $this->table = "clientes";
        $this->primaryKey = "id_cliente";

        $this->joins = array(
            array(
                'table'=>'clientes_tipos',
                'id'=>'id_cliente_tipo',
                'fk'=>'id_tipo',
                'columns'=>array(
                    array(
                        'name'=>'nombre',
                        'alias'=>'nombre_tipo'
                    ),
                ),
            ),
            array(
                'table'=>'puntos_de_venta',
                'id'=>'id_pdv',
                'fk'=>'id_pdv',
                'columns'=>array(
                    array(
                        'name'=>'nombre',
                        'alias'=>'nombre_pdv'
                    ),
                ),
            ),
        );
    }

    protected array $fillable = [
        'nombre',
        'direccion',
        'telefono',
        'email',
        'id_tipo',
        'id_pdv',
        'datos_facturacion',
        'notas',
    ];
}
```

○ Descripción del Modelo Cliente:

- Tabla y Clave Primaria: clientes, id\_cliente
- Uniones: clientes\_tipos: Une la tabla clientes con clientes\_tipos para obtener el nombre del tipo de cliente. puntos\_de\_venta: Une la tabla clientes con puntos\_de\_venta para obtener el nombre del punto de venta.
- Campos Rellenables: nombre, direccion, telefono, email, id\_tipo, id\_pdv, datos\_facturacion, notas.

○ Lista de Modelos y Propósitos

- Cliente.php: Almacena información sobre los clientes.
- ClienteTipo.php: Clasifica a los clientes en diferentes tipos.



- Departamento.php: Almacena información sobre los departamentos dentro de la empresa.
- Documento.php: Gestiona documentos importantes relacionados con empleados y vehículos.
- DocumentoTipo.php: Clasifica los tipos de documentos.
- Empleado.php: Almacena información sobre los empleados.
- EquipoMontaje.php: Gestiona los equipos de montaje.
- Fabricacion.php: Registra la producción de las ventanas.
- Factura.php: Documenta las transacciones de venta.
- Incidencia.php: Gestiona problemas o anomalías surgidas durante las obras o postventa.
- IncidenciaFoto.php: Almacena fotos relacionadas con incidencias.
- Material.php: Clasifica los materiales utilizados en fabricaciones y pedidos.
- Medicion.php: Detalla las mediciones realizadas para las obras.
- Montaje.php: Registra la información sobre la instalación de productos en la obra.
- Obra.php: Gestiona los proyectos de fabricación e instalación de ventanas.
- Pedido.php: Gestiona las órdenes de compra de materiales.
- PedidoTipo.php: Clasifica las órdenes de compra.
- Proveedor.php: Almacena información sobre los proveedores de materiales o servicios.
- PuntoDeVenta.php: Representa las ubicaciones físicas o virtuales donde los clientes realizan transacciones.
- Usuario.php: Gestiona las cuentas de acceso al sistema para empleados.
- Vehiculo.php: Almacena información sobre los vehículos utilizados por la empresa.
- Un modelo tiene características particulares:
  - Usuario.php: Extiende de Authenticatable y tiene un campo hidden para ocultar la contraseña.

- **Controllers:** Los controladores en el proyecto son responsables de gestionar las acciones y operaciones relacionadas con los modelos de datos. Cada modelo tiene un controlador correspondiente que maneja las solicitudes HTTP y realiza las operaciones necesarias en la base de datos utilizando el ORM proporcionado por la clase base Model. Los controladores extienden de la clase base Controller. Los controladores suelen incluir middleware de autenticación y autorización para garantizar que solo los usuarios autorizados puedan acceder a ciertas rutas y realizar operaciones específicas. Cada controlador sigue una estructura estándar, que incluye las siguientes funciones principales:
  - **Index:** Esta función maneja las solicitudes GET y devuelve una respuesta en formato JSON con un array que contiene todos los registros del modelo correspondiente. En algunos casos, la función index puede recibir parámetros de consulta (query params) para filtrar y personalizar los resultados, como filtrar obras por tipo de material.
  - **Store:** La función store maneja las solicitudes POST para crear nuevos registros. Valida los datos recibidos en la solicitud y, si son válidos, crea un nuevo objeto en la base de datos utilizando el ORM. Después de insertar el nuevo objeto, la función devuelve el objeto creado en formato JSON. En algunos casos, store realiza transformaciones en los datos para asegurarse de que se insertan en el formato correcto.
  - **Show:** La función show maneja las solicitudes GET para recuperar los datos de un registro específico. Utiliza el dependency injection para obtener el objeto correspondiente y devuelve sus datos en formato JSON. Algunos controladores tienen funciones show especiales que devuelven archivos u otros datos relacionados con el modelo principal.
  - **Update:** La función update maneja las solicitudes PUT para actualizar los datos de un registro existente. Valida los datos recibidos en la solicitud y, si son válidos, actualiza el objeto correspondiente en la base de datos. La función devuelve el objeto actualizado en formato JSON.
  - **Destroy:** La función destroy maneja las solicitudes DELETE para marcar un registro como inactivo. En lugar de eliminar el registro de la base de datos, el ORM cambia un campo active para indicar que el registro ha sido desactivado. Esto permite mantener un historial de los registros sin eliminarlos físicamente.

- Además de estas funciones principales, algunos controladores pueden tener funciones adicionales para manejar casos de uso específicos, como mostrar registros relacionados o gestionar archivos.
- Ejemplo Detallado: Controlador ClienteController

```
<?php
namespace App\Controllers;

use api\Http\Controller;
use api\Http\Request;
use App\Middlewares\AuthMiddleware;
use App\Models\Cliente;
use App\Models\Obras;

class ClienteController extends Controller{

    public function __construct(){
        $this->setMiddlewares([AuthMiddleware::class]);
    }

    public function index(){
        return json(Cliente::allJoined() ?? []);
    }

    public function store(Request $request){
        $data = $request->validate([
            'nombre' => ['max_length:50', 'required'],
            'direccion' => ['max_length:255'],
            'telefono' => ['phone_number'],
            'email' => ['email'],
            'id_tipo' => ['required', 'number'],
            'id_pdv' => ['required', 'number'],
            'datos_facturacion' => ['max_length:50'],
            'notas' => ['max_length:500']
        ]);

        $data['nombre'] = ucwords(strtolower(trim($data['nombre'])));

        Cliente::create($data);

        $cliente = Cliente::lastInsert();

        return redirect('/clientes/'.$cliente->id_cliente);
    }

    public function show(Cliente $cliente){
        return json($cliente->toArray());
    }

    public function showObras(Cliente $cliente){
        $idObras = app()->database->statement("select id_obra from obras where id_cliente=?", [
            $idObras = array_map(fn($array) => $array['id_obra'], $idObras);

        $obras = Obras::find($idObras);

        return json($obras);
    }

    public function update(Cliente $cliente, Request $request){
        $data = $request->validate([
            'nombre' => ['max_length:50', 'required'],
            'direccion' => ['max_length:255'],
            'telefono' => ['phone_number'],
            'email' => ['email'],
            'id_tipo' => ['required', 'number'],
            'id_pdv' => ['required', 'number'],
            'datos_facturacion' => ['max_length:50'],
            'notas' => ['max_length:500']
        ]);

        $data['nombre'] = ucwords(strtolower(trim($data['nombre'])));

        $cliente->nombre = $data['nombre'];
        $cliente->direccion = $data['direccion'];
        $cliente->telefono = $data['telefono'];
        $cliente->email = $data['email'];
        $cliente->id_tipo = $data['id_tipo'];
        $cliente->id_pdv = $data['id_pdv'];
        $cliente->datos_facturacion = $data['datos_facturacion'];
        $cliente->notas = $data['notas'];

        $cliente->update();

        return redirect('/clientes/'.$cliente->id_cliente);
    }

    public function destroy(Cliente $cliente){
        $cliente->delete();
        session()->flash('alert', 'Cliente '.$cliente->nombre borrado exitosamente');
        return redirect('/clientes');
    }
}
```

- Descripción del Controlador ClienteController:
  - Constructor: Sobrescribe el constructor de Controller para incluir middleware de autenticación utilizando setMiddlewares.
  - Index: Recupera y devuelve todos los registros de clientes con sus datos relacionados en formato JSON. Utiliza Cliente::allJoined() para incluir uniones con otras tablas.
  - Store: Valida y procesa los datos de la solicitud POST para crear un nuevo cliente. Los datos se transforman (por ejemplo, capitalizando el nombre del cliente) antes de crear el nuevo registro en la base de datos. La función redirige al endpoint del cliente recién creado.
  - Show: Devuelve los datos de un cliente específico en formato JSON, utilizando el dependency injection para resolver el cliente.
  - ShowObras: Recupera y devuelve todas las obras relacionadas con un cliente específico.
  - Update: Valida y procesa los datos de la solicitud PUT para actualizar un cliente existente. Los datos se transforman antes de actualizar el

registro en la base de datos. La función redirige al endpoint del cliente actualizado.

- Destroy: Marca un cliente como inactivo utilizando el ORM y redirige a la lista de clientes. Se muestra un mensaje de confirmación en la sesión.
- Lista de Controladores y Propósitos
  - Auth\LoginController: Gestiona el inicio y cierre de sesión de los usuarios.
  - ClienteController: Gestiona las operaciones relacionadas con los clientes.
  - ClienteTipoController: Gestiona los tipos de clientes.
  - DepartamentoController: Administra los departamentos de la empresa.
  - DocumentoController: Gestiona los documentos de empleados y vehículos.
  - DocumentoTipoController: Administra los tipos de documentos.
  - EmpleadoController: Gestiona la información de los empleados.
  - EquipoMontajeController: Gestiona los equipos de montaje y sus empleados.
  - FabricacionController: Administra las operaciones de fabricación.
  - FacturaController: Gestiona las facturas emitidas.
  - IncidenciaController: Administra las incidencias reportadas.
  - IncidenciaAccionController: Gestiona las acciones tomadas en respuesta a incidencias.
  - IncidenciaFotoController: Administra las fotos relacionadas con incidencias.
  - MaterialController: Gestiona los materiales utilizados en fabricaciones y pedidos.
  - MedicionController: Administra las mediciones realizadas para las obras.
  - MontajeController: Gestiona las operaciones de montaje.
  - ObraController: Administra los proyectos de fabricación e instalación de ventanas.
  - PedidoController: Gestiona las órdenes de compra de materiales.
  - PedidoTipoController: Administra los tipos de pedidos.

- ProveedorController: Gestiona la información sobre los proveedores.
- PuntoDeVentaController: Representa las ubicaciones físicas o virtuales donde los clientes realizan transacciones.
- UsuarioController: Gestiona las cuentas de acceso al sistema para empleados.
- VehiculoController: Administra los vehículos utilizados por la empresa.
- WelcomeController: Proporciona una respuesta de bienvenida y verifica el estado de autenticación del usuario.
- Particularidades
  - LoginController: Utiliza la clase Hasher para verificar las credenciales de usuario durante el inicio de sesión y maneja el estado de autenticación del usuario.
  - DocumentoController: Tiene funciones especiales para gestionar la subida y visualización de archivos PDF.
  - IncidenciaController: Gestiona múltiples estados de incidencias y tiene funciones adicionales para mostrar fotos relacionadas con una incidencia específica.
  - ObraController: Incluye funciones para filtrar obras por diferentes materiales y gestionar proyectos de fabricación relacionados con la obra.
- **Middlewares:** Los middlewares en el proyecto actúan como filtros que se ejecutan antes de las funciones de los controladores que los implementan. Su propósito principal es manejar tareas comunes como la autenticación, la autorización, la validación de solicitudes y la manipulación de respuestas. Los middlewares permiten la separación de responsabilidades y promueven la reutilización de código, garantizando que las operaciones críticas, como la verificación de permisos y la autenticación del usuario, se realicen de manera consistente en toda la aplicación.
  - Descripción del AuthMiddleware:
    - Función handle: Esta función se ejecuta para cada solicitud que pasa por el middleware. Primero verifica si el usuario está autenticado utilizando `Auth::isGuest()`. Si el usuario no está autenticado, lanza una excepción `AuthException`. Luego, llama a la función `checkPermission` para verificar si el usuario tiene los permisos necesarios para realizar la acción solicitada. Si no tiene permisos, lanza una excepción `PermissionException`. En este caso por ahora la función

checkPermission devuelve siempre true ya que la implementación de control de permisos está preparada actualmente pero se implementará a futuro.

- Función checkPermission: Esta función verifica los permisos del usuario para la acción solicitada. Obtiene la acción y la tabla de la ruta de la solicitud. Determina el rol del usuario autenticado. Si el rol del usuario es 2, obtiene el departamento del usuario y verifica si tiene los permisos necesarios consultando la tabla usuarios\_permisos en la base de datos. Si el rol del usuario es 1 (superusuario), permite automáticamente la acción. Devuelve true si el usuario tiene los permisos necesarios, false en caso contrario.
- **Providers:** Son componentes que se utilizan para registrar servicios y funcionalidades personalizadas dentro del framework. Estos providers permiten cargar y configurar servicios esenciales que son utilizados a lo largo de la aplicación. En este proyecto, se han implementado dos providers principales:
  - RouteServiceProvider: Se encarga de cargar las rutas de la aplicación desde una carpeta especificada, asegurando que todas las rutas definidas estén disponibles para la API.
    - Función registerServices: Esta función se ejecuta para registrar los servicios relacionados con las rutas de la aplicación. Utiliza Route::load para cargar todas las rutas definidas en la carpeta especificada (App::\$root."/routes"). Esto asegura que todas las rutas estén disponibles para ser utilizadas por la API. La función es invocada durante el proceso de inicialización de la aplicación, garantizando que las rutas estén configuradas antes de que se procesen las solicitudes.
  - RuleServiceProvider: Registra y carga validaciones personalizadas que se utilizan en la API para validar los datos de las solicitudes.
    - Función registerServices: Esta función se ejecuta para registrar los servicios relacionados con las reglas de validación personalizadas. Primero, carga las reglas de validación predeterminadas utilizando Rule::loadDefaultRules(). Luego, utiliza Rule::load para cargar un array de clases de reglas personalizadas. Estas clases definen validaciones

específicas que se utilizan en la API para validar los datos de las solicitudes.

- Las reglas personalizadas incluyen validaciones para contraseñas, números de teléfono, fechas, estados específicos para diversas entidades (mediciones, obras, fabricaciones, incidencias, etc.), y tipos de servicios.
  - La función garantiza que todas las reglas de validación estén disponibles y configuradas correctamente antes de que se procesen las solicitudes que requieren validación.
- 
- **Rules:** Son componentes que aseguran que los datos recibidos en las solicitudes sean válidos y cumplan con ciertos criterios específicos antes de ser procesados. Estas reglas implementan la interfaz `ValidationRule`, que define dos métodos principales: `message` y `isValid`.
    - Método `message`: Devuelve el mensaje de error que se debe mostrar si la validación falla.
    - Método `isValid`: Contiene la lógica de validación que verifica si el valor de un campo específico es válido o no.
    - Las reglas de validación se utilizan para comprobar diversos tipos de datos, como fechas, números de teléfono, estados específicos para diferentes entidades y contraseñas. Al implementar estas reglas, se garantiza que los datos ingresados en la aplicación sean consistentes y correctos, mejorando la fiabilidad y la seguridad del sistema.
    - `Boolean.php`: Valida que el valor de un campo sea `true` o `false`.
    - `Date.php`: Valida que el valor de un campo sea una fecha con formato `YYYY-MM-DD`.
    - `DateTime.php`: Valida que el valor de un campo sea una fecha y hora con formato `YYYY-MM-DD HH:MM:SS`.
    - `EstadoFabricacion.php`: Valida que el valor de un campo sea uno de los estados permitidos para una fabricación (Creado, Pendiente de fabricar, Fabricado, Falta).
    - `EstadoFactura.php`: Valida que el valor de un campo sea uno de los estados permitidos para una factura (Pagado, Emitida).

- EstadoIncidencia.php: Valida que el valor de un campo sea uno de los estados permitidos para una incidencia (Solicitada, Citada, Falta, Finalizada).
- EstadoMedicion.php: Valida que el valor de un campo sea uno de los estados permitidos para una medición (Pendiente, Citado, Realizado, Presupuestado, Otro).
- EstadoMontaje.php: Valida que el valor de un campo sea uno de los estados permitidos para un montaje (No finalizado, Finalizado).
- EstadoObra.php: Valida que el valor de un campo sea uno de los estados permitidos para una obra (Solicitado, Presupuestado, Aceptado, Fabricado, Finalizado).
- EstadoPedido.php: Valida que el valor de un campo sea uno de los estados permitidos para un pedido (Pedido, Completo, Falta).
- Password.php: Valida que una contraseña cumpla con los criterios de longitud y contenido (8-20 caracteres, al menos una mayúscula, una minúscula y un número).
- PhoneNumber.php: Valida que el valor de un campo sea un número de teléfono con formato correcto (empieza con 6, 7, 8 o 9 y tiene 9 dígitos).
- TipoServicio.php: Valida que el valor de un campo sea uno de los tipos de servicio permitidos (Montaje, Recogida en Fábrica, Porte)

Todo este código descrito de la API se encuentra dentro de la carpeta App, pero hay mas carpetas con codigo relevante a nivel de la carpeta de app, y src (la cual contiene el codigo del framework), podemos encontrar diferentes carpetas en el proyecto:

- **config:** la cual contiene archivos que recogen variables de entorno del archivo .env y en caso de que no existan asignan un valor por defecto.
- **public:** contiene un archivo index.php el cual es el punto de entrada a la API, es donde se debe arrancar el servidor.
- **routes:** se encuentran todos los archivos que crean las rutas para que la API, tienen todas las rutas para gestionar el CRUD, cada ruta llama a una funcion del correspondiente Controller, estas rutas pueden pasar argumentos y el dependencyinjection se encarga de crear los objetos correspondientes, los archivos con rutas son los siguientes: api.php clientes.php clientes\_tipos.php departamentos.php documentos.php documentos\_tipos.php empleados.php equipos\_montaje.php fabricaciones.php facturas.php incidencias.php incidencias\_acciones.php incidencias\_fotos.php materiales.php mediciones.php montajes.php obras.php



pedidos.php pedidos\_tipos.php proveedores.php puntos\_de\_venta.php usuarios.php  
usuarios\_permisos.php vehiculos.php

- **storage:** carpeta que guarda los archivos que se envían
- **.env:** archivo en el cual se configuran las variables de entorno para la configuración del framework y de la API.

### 5.2.3. FRONT REACT

La parte frontal de la aplicación está desarrollada con React, utilizando una estructura típica de una SPA. La aplicación está diseñada para ser dinámica y receptiva, proporcionando una experiencia de usuario fluida. El proyecto está estructurado de la siguiente manera:

- **public/:** Contiene activos públicos como imágenes e íconos que se utilizan en la aplicación.
- **src/:** La carpeta principal que contiene todos los archivos fuente de la aplicación React.
  - **assets/img:** Imágenes e íconos adicionales.
  - **components/:** Contiene todos los componentes React reutilizables organizados en subcarpetas por funcionalidad.
    - **administracion/:** Componentes relacionados con la administración de la empresa.
    - **clientes/:** Componentes para la gestión de clientes.
    - **contabilidad/:** Componentes para la gestión de contabilidad y facturas.
    - **incidencias/:** Componentes para la gestión de incidencias.
    - **mediciones/:** Componentes para la gestión de mediciones.
    - **menu/:** Componentes para la navegación del menú.
    - **montajes/:** Componentes para la gestión de montajes.
    - **obras/:** Componentes para la gestión de obras.
    - **produccion/:** Componentes para la gestión de producción.
  - **contexts/:** Define los contextos React para la gestión del estado global.
    - **TableContext.jsx:** Contexto para funciones de la tabla.
  - **css/:** Archivos de estilo CSS organizados por funcionalidad.
  - **App.jsx:** Componente principal que contiene las rutas de la aplicación.
  - **main.jsx:** Punto de entrada principal para renderizar la aplicación en el DOM.
- **index.html:** El punto de entrada HTML de la aplicación.

- **Dependencias:** El proyecto utiliza varias dependencias y herramientas de desarrollo para mejorar la funcionalidad y el flujo de trabajo:
  - React: Biblioteca principal para construir la interfaz de usuario.
  - React Router Dom: Para la gestión de la navegación en la aplicación.
  - Day.js: Librería para manipulación de fechas.
  - React Big Calendar: Para implementar un calendario interactivo.
  - Tanstack React Table: Para la gestión avanzada de tablas de datos.
  - Vite: Como empaquetador y servidor de desarrollo para una configuración rápida y eficiente.

En el siguiente apartado se describirán los componentes a mayor detalle:

- **main.jsx:** establece el punto de entrada de la aplicación React, montando el componente App en el elemento con el id root del DOM. Utiliza BrowserRouter para habilitar el enrutamiento del navegador y envuelve todo en React.StrictMode para proporcionar comprobaciones adicionales en el modo de desarrollo. Este archivo configura la estructura básica y el enrutamiento de la aplicación, asegurando que todas las rutas definidas en App funcionen correctamente.
- **app.jsx:** es el corazón de la lógica de enrutamiento de la aplicación. Utiliza react-router-dom para definir las rutas y componentes que se deben renderizar en función de la URL actual:
  - **Importaciones principales:** Se importan Routes, Route, y Navigate de react-router-dom para configurar las rutas de la aplicación.
  - **Componentes:** Se importan todos los componentes necesarios para las diferentes rutas, organizados por funcionalidad.
  - **Contextos y estilos:** Se importan TableContext para la gestión del estado global relacionado con las tablas y varios archivos CSS para los estilos de la aplicación.
  - **Componente Auth:**
    - Estado del usuario: Utiliza useState para gestionar el estado del usuario.
    - Verificación de autenticación: Utiliza useEffect para realizar una solicitud a la API y verificar si el usuario está autenticado.
    - Renderizado condicional: Muestra un mensaje de carga mientras se verifica la autenticación, renderiza el componente Main si el usuario

está autenticado, o redirige al usuario a la página de inicio de sesión (/login) si no lo está.

- **Gestión de temas:** Utiliza el estado theme para manejar el tema actual (claro u oscuro). El useEffect se asegura de aplicar la clase correspondiente al tema en el elemento raíz del documento.
- **Definición de Rutas:** Define una serie de rutas utilizando Routes y Route de react-router-dom. Cada ruta especifica un path y el componente que debe renderizarse (element) cuando esa ruta está activa. Usa el componente Auth para envolver rutas que requieren autenticación, pasando el componente específico y el titulo como props.
- **Autenticación y Contexto:** El componente Auth se encarga de verificar la autenticación del usuario y proporciona el contexto necesario para la gestión de tablas (TableContext).

Dentro de la carpeta components podemos encontrar las componentes que componen la aplicación:

- **Main.jsx:** es un componente de presentación que organiza y estructura la interfaz principal de la aplicación, incluyendo la barra lateral de navegación (SideBar), el menú superior (TopMenu) y el contenido principal que se renderiza basado en los parámetros recibidos. Acepta un objeto parametros como argumento. Este objeto contiene varios parámetros que se pasan a los componentes hijos.
- **SideBar.jsx:** organiza y muestra la barra lateral de navegación de la aplicación. Utiliza los hooks de React para gestionar el estado y los efectos secundarios necesarios para resaltar el enlace activo y manejar la navegación. Este diseño modular y organizado facilita la navegación por la aplicación y permite a los usuarios acceder rápidamente a las diferentes secciones de la misma.
- **TopMenu.jsx:** es responsable de renderizar el menú superior de la aplicación. Este componente muestra el titulo de la página actual, una barra de búsqueda condicional, la hora actual y la información del usuario autenticado, incluyendo su nombre completo y departamento.
  - **Hora Actual:** Incluye una función getHora que obtiene la hora actual y la formatea en un objeto con las propiedades hora y minuto.

- Al inicializar realiza una solicitud a la API para obtener la información del usuario autenticado y su departamento.
  - Si searchable es true, se muestra un campo de entrada de texto para la búsqueda.
- 
- **OpenIn.jsx:** Componente básico con un enlace en un icono de “abrir en”.
  
  - **LogIn.jsx:** es responsable de renderizar el formulario de inicio de sesión y gestionar el proceso de autenticación del usuario. Este componente maneja la validación del formulario, la interacción con la API de autenticación y la gestión de errores.
    - **Estado y Referencias:** Utiliza useRef para crear una referencia al formulario y useNavigate de react-router-dom para navegar programáticamente tras el inicio de sesión exitoso.
    - **Expresiones Regulares:** Define varias expresiones regulares (emailRegex, passwordRegex) para validar el formato del correo electrónico y la contraseña.
    - **Manejo del Envío del Formulario:** La función handleSubmit se encarga de manejar el evento de envío del formulario. Convierte los datos del formulario en un objeto, realiza la validación, y si los datos son válidos, envía una solicitud POST a la API de inicio de sesión. En caso de error, muestra mensajes de error en los campos correspondientes.
    - **Validación:** La función validate comprueba que el correo electrónico tenga un formato válido y que la contraseña cumpla con los criterios establecidos (longitud y contenido).
    - **Gestión de Respuestas de la API:** La función manageResponse maneja las respuestas de la API, mostrando alertas en caso de credenciales incorrectas o datos no válidos, y redirigiendo al usuario a la página principal en caso de éxito.
    - **Limpieza de Errores:** La función clearErrores elimina cualquier mensaje de error previo antes de realizar una nueva validación.
    - **Renderizado:** El componente renderiza una estructura de formulario sencilla con campos para el correo electrónico y la contraseña, y un botón de envío.

También incluye una sección lateral de bienvenida y el logotipo de la aplicación.

- **LogOut.jsx:** es responsable de cerrar la sesión del usuario y redirigirlo a la página de inicio de sesión. Realiza una solicitud a la API para cerrar la sesión y gestiona el estado del cierre de sesión para proporcionar retroalimentación al usuario. Este componente asegura que el proceso de cierre de sesión sea manejado de manera efectiva y el usuario sea redirigido adecuadamente.
- **Table.jsx:** es un componente reutilizable que renderiza una tabla con capacidades avanzadas de filtrado, paginación y ordenación, utilizando la biblioteca @tanstack/react-table. Este componente es esencial para mostrar listas de registros en diferentes secciones de la aplicación, como clientes, obras, incidencias, etc:
  - **Estado y Contexto:** Utiliza useState para manejar el estado de la ordenación (sorting) y la paginación (pagination). Utiliza useContext para acceder al contexto global de la tabla (TableContext), permitiendo la gestión del filtro global (searchInput).
  - **Configuración de la Tabla:** El hook useReactTable se utiliza para configurar la tabla con los datos (data), las columnas (columns) y varios modelos de la tabla como getCoreRowModel, getPaginationRowModel, getSortedRowModel, y getFilteredRowModel. La tabla también gestiona el estado de paginación, ordenación y filtrado global.
  - **Efectos:** Un useEffect inicializa la capacidad de búsqueda (searchable) basada en el prop search.
  - **Renderizado de la Tabla:** La tabla se renderiza en un elemento table con un thead y un tbody. El encabezado (thead) se configura para manejar la ordenación de columnas y, opcionalmente, incluye un botón "Crear" para agregar nuevos registros. El cuerpo (tbody) muestra las filas de datos y, opcionalmente, incluye botones para ver detalles y eliminar registros. La tabla también maneja la paginación, mostrando botones para navegar entre páginas si hay más de una página de resultados.

- **Acciones de la Tabla:** Las acciones (actions) permiten la creación, visualización de detalles y eliminación de registros. El componente maneja las solicitudes de eliminación y confirma al usuario antes de eliminar un registro.

La carpeta de componentes contiene subcarpetas con todos los componentes necesarios para la aplicación existen distintos tipos de componentes para realizar las operaciones CRUD:

- **Componentes de listado:** Los componentes que listan registros en la aplicación se encargan de mostrar datos en una tabla utilizando el componente reusable Table. Estos componentes están diseñados para obtener datos de la API, gestionarlos y renderizarlos de manera eficiente, proporcionando funcionalidades de búsqueda, ordenación y acciones específicas para cada registro:
  - **Estado y Efectos:** Utilizan el hook useState para mantener el estado de los datos que se van a listar. Utilizan el hook useEffect para realizar una solicitud fetch a la API cuando el componente se monta. La respuesta de la API se guarda en el estado correspondiente.
  - **Definición de Columnas:** Definen las columnas de la tabla especificando los encabezados (header) y las claves de acceso (accessorKey) para mapear los datos obtenidos a las columnas correspondientes de la tabla. Las columnas pueden incluir varios campos de información relevante para el tipo de datos que se está listando, como nombres, direcciones, teléfonos, correos electrónicos, etc.
  - **Acciones de la Tabla:** Configuran las acciones disponibles en la tabla, que pueden incluir: create: Ruta para crear un nuevo registro. detail: Ruta para ver los detalles de un registro específico. delete: Ruta para eliminar un registro específico. id: Clave única para identificar cada registro en las acciones de detalle y eliminación.
  - **Renderizado de la Tabla:** Utilizan el componente Table para renderizar los datos en una tabla. Pasan los datos (data), las columnas (columns), y las acciones (actions) como props al componente Table. La tabla incluye funcionalidades avanzadas como búsqueda, ordenación, paginación, y botones para ver detalles y crear nuevos registros.

- **Componentes de crear:** Los componentes de creación en la aplicación están diseñados para permitir a los usuarios añadir nuevos registros a la base de datos. Estos componentes muestran un formulario con los campos necesarios, validan los datos introducidos y envían una solicitud a la API para guardar el nuevo registro:
  - **Estado y Efectos:** Utilizan useState para manejar el estado local de los datos necesarios para el formulario, como listas de opciones (por ejemplo, clientes, materiales). Utilizan useEffect para realizar solicitudes fetch a la API y obtener datos necesarios para llenar los campos de selección en el formulario.
  - **Formulario de Creación:** Renderizan un formulario que incluye campos para todos los atributos del objeto que se va a crear. Cada campo del formulario está asociado a un estado y puede incluir selectores, campos de texto, fechas, números, etc.
  - **Validación de Datos:** Implementan funciones de validación para asegurar que los datos introducidos por el usuario cumplen con los criterios necesarios (por ejemplo, formatos de fecha, longitud de texto, campos obligatorios). En caso de errores de validación, se muestran mensajes de error cerca de los campos correspondientes.
  - **Manejo del Envío del Formulario:** La función handleSubmit se encarga de manejar el evento de envío del formulario. Recoge los datos del formulario, los valida y, si son válidos, crea un objeto FormData que se envía mediante una solicitud fetch a la API para crear el nuevo registro. Si la solicitud es exitosa, redirige al usuario a la página correspondiente, normalmente la lista de registros.
- **Componentes de modificar/detalle:** Los componentes de modificación o detalle están diseñados para mostrar y permitir la edición de los detalles de un registro existente en la base de datos. Estos componentes recuperan los datos actuales del registro, los muestran en un formulario pre-relleno y permiten al usuario modificar los datos y guardarlos nuevamente en la base de datos.
  - **Estado y Efectos:** Utilizan useState para mantener el estado de los datos del registro, así como listas de opciones necesarias para los campos de selección (por ejemplo, clientes, materiales). Utilizan useEffect para realizar solicitudes fetch a la API para obtener los datos actuales del registro y las listas de opciones necesarias cuando el componente se monta.

- **Formulario de Modificación:** Renderizan un formulario pre-relleno con los datos actuales del registro. Cada campo del formulario está vinculado al estado correspondiente y se actualiza con las entradas del usuario.
  - **Validación de Datos:** Implementan funciones de validación para asegurar que los datos introducidos por el usuario cumplen con los criterios necesarios (por ejemplo, formatos de fecha, longitud de texto, campos obligatorios). En caso de errores de validación, se muestran mensajes de error cerca de los campos correspondientes.
  - **Manejo del Envío del Formulario:** La función `handleSubmit` se encarga de manejar el evento de envío del formulario. Recoge los datos del formulario, los valida y, si son válidos, crea un objeto `FormData` que se envía mediante una solicitud `fetch` a la API para actualizar el registro. Si la solicitud es exitosa, redirige al usuario a la página correspondiente, normalmente la lista de registros.
  - **Eliminación de Registros:** Implementan una función para eliminar el registro actual, que envía una solicitud `fetch DELETE` a la API y redirige al usuario a la lista de registros tras una eliminación exitosa.
- 
- Componentes de menú: están diseñados para organizar y mostrar enlaces de navegación estructurados. Estos componentes facilitan la creación de menús dinámicos, ya sea en forma de listas simples o categorizadas, y permiten a los usuarios acceder fácilmente a diferentes secciones de la aplicación.
    - **Menu.jsx:** Este componente recibe un array `menu` como prop, que contiene objetos con la información de cada enlace (nombre, link, imagen). Utiliza el componente `MenuLink` para renderizar cada enlace, pasando las propiedades necesarias. Renderiza los enlaces en un contenedor `div` con la clase `menu-container`.
    - **MenuCategorizado.jsx:** Similar a `Menu`, pero organiza los enlaces en categorías. Recibe un array `menu` que contiene objetos de categoría, cada uno con un título y un array de enlaces. Renderiza las categorías con un encabezado (`h2`) y utiliza `MenuLink` para renderizar los enlaces dentro de cada categoría.
    - **MenuLink.jsx:** Este componente se encarga de crear un enlace estilizado para el menú. Utiliza `Link` de `react-router-dom` para la navegación. Aplica estilos de



fondo a los enlaces utilizando las imágenes proporcionadas en las props. Renderiza el texto del enlace dentro de un contenedor div con la clase menu-link.

- **Ejemplos de Uso:** MenuObras.jsx: Define un menú específico para la sección de obras, con enlaces a varias categorías de obras. Pasa el array menu al componente Menu para renderizar los enlaces. MenuHome.jsx: Define un menú categorizado para la página principal, con enlaces agrupados en categorías como "CRM", "PRODUCCIÓN", "ADMINISTRACIÓN" y "CONTABILIDAD". Pasa el array menu al componente MenuCategorizado para renderizar los enlaces organizados por categorías.

Para convertir la aplicación en una Progressive Web App (PWA), se utiliza la configuración de Vite junto con el plugin vite-plugin-pwa.

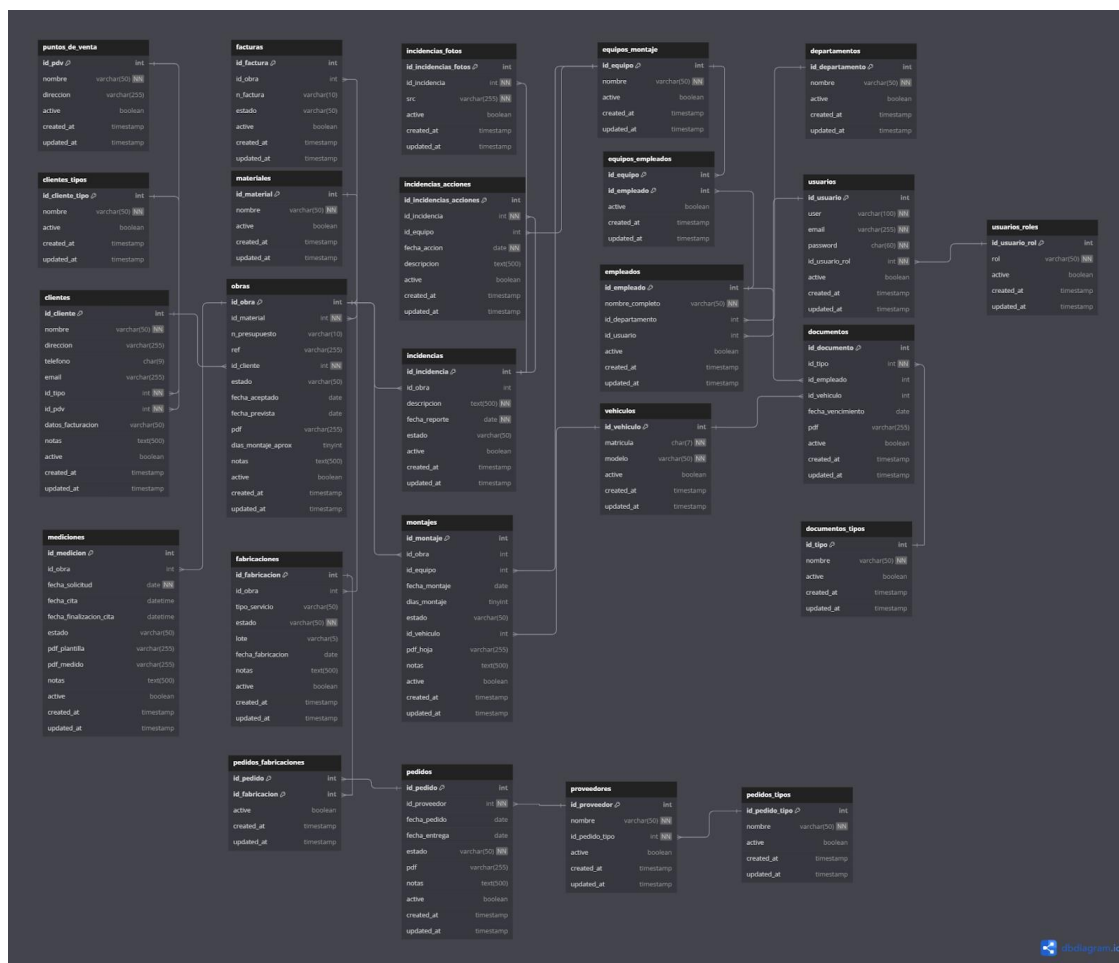
#### Vite:

- defineConfig de vite se utiliza para exportar la configuración.
- react se incluye como plugin para habilitar el soporte de React.
- svgr se utiliza para importar y utilizar archivos SVG como componentes de React.
- VitePWA se importa desde vite-plugin-pwa y se incluye en el array de plugins.

El objeto manifest dentro de la configuración define las propiedades esenciales de la PWA:

- display: Define cómo se muestra la aplicación al usuario. standalone permite que la aplicación se comporte como una aplicación nativa sin barra de navegador.
- display\_override: Define alternativas de visualización, como window-controls-overlay para dispositivos específicos.
- name y short\_name: El nombre completo y el nombre corto de la aplicación, que se muestran en los dispositivos.
- description: Breve descripción de la aplicación.
- theme\_color: Color del tema de la aplicación.
- background\_color: Color de fondo de la pantalla de inicio y de carga.
- icons: Array de objetos que especifican los íconos utilizados para la aplicación en diferentes tamaños y propósitos.
- src: Ruta del ícono.
- sizes: Tamaño del ícono.
- type: Tipo de archivo del ícono.
- purpose: Propósito del ícono, como maskable para permitir formas adaptables.

### 5.3. Diseño de tablas de la base de datos



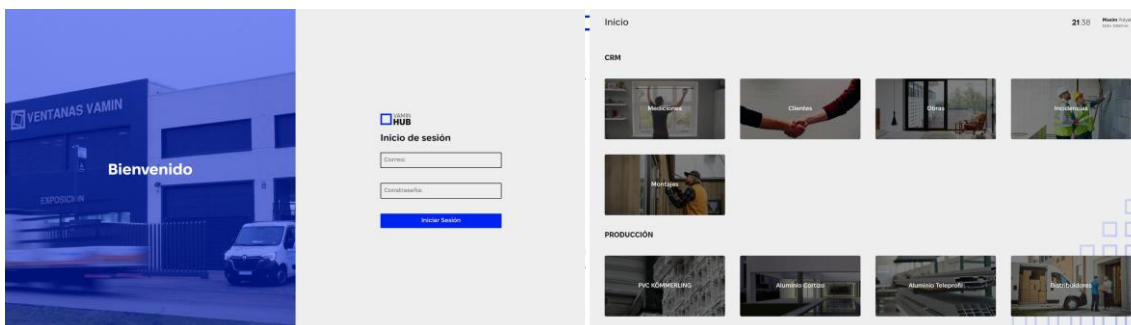
## 6. MANUAL DE USUARIO

### Inicio de sesión

- Abre el Navegador: Utiliza cualquier navegador web moderno (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) para acceder a la URL de Vamin HUB.
- Accede a la Página de Inicio de Sesión: Dirígete a la URL proporcionada por la empresa para acceder al sistema Vamin HUB.
- Introduce tus Credenciales: Correo: Escribe tu dirección de correo electrónico registrada. Contraseña: Escribe tu contraseña.
- Iniciar Sesión: Haz clic en el botón "Iniciar Sesión".

### Dashboard

- Una vez hayas iniciado sesión, serás redirigido al Dashboard. Aquí podrás ver accesos rápidos a las principales secciones del sistema.



### Gestión de Clientes

Accede a la Sección de Clientes desde el menú lateral, selecciona la opción "Clientes".

Añadir un Nuevo Cliente:

- Haz clic en "Añadir Cliente".
- Completa el formulario con los datos del cliente (nombre, dirección, contacto, etc.).
- Haz clic en "Guardar" para añadir el nuevo cliente.

Modificar un Cliente Existente:

- Selecciona el cliente que deseas modificar de la lista.
- Haz clic en el botón de la primera columna y actualiza los datos necesarios.

- Haz clic en "Guardar" para aplicar los cambios.

Eliminar un Cliente:

- Selecciona el cliente que deseas eliminar.
- Haz clic en "Eliminar" y confirma la acción.

## Mediciones

Accede a la Sección de Mediciones:

- Desde el menú lateral, selecciona "Mediciones".

Crear una Nueva Solicitud de Medición:

- Haz clic en "Crear Medición".
- Completa el formulario con los detalles de la medición.
- Haz clic en "Guardar" para añadir la nueva solicitud.

Visualizar y Modificar Mediciones Existentes:

- Selecciona una medición de la lista para ver sus detalles.
- Haz clic en el botón de la primera columna para modificar los datos y guarda los cambios.

## Obras

Accede a la Sección de Obras:

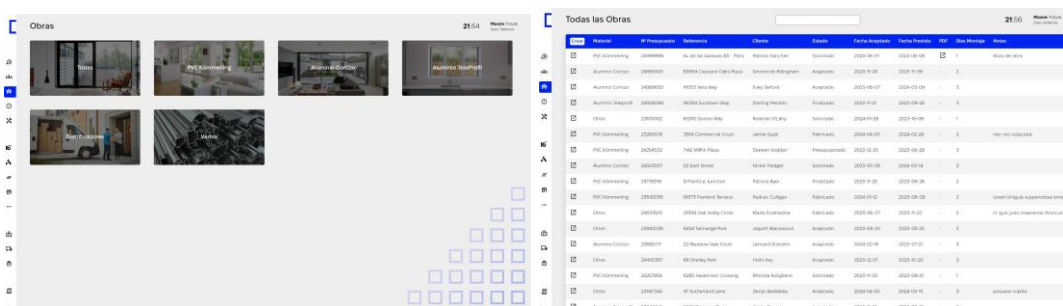
- Desde el menú lateral, selecciona "Obras".

Visualiza la Lista de Obras:

- Navega por la lista para ver todas las obras actuales.

Añadir o Modificar Detalles de una Obra:

- Selecciona una obra de la lista y haz clic en el botón de la primera columna.
- Actualiza la información necesaria y guarda los cambios.
- Selecciona una obra de la lista y haz clic en el botón de la primera columna.



## Incidencias

Accede a la Sección de Incidencias:

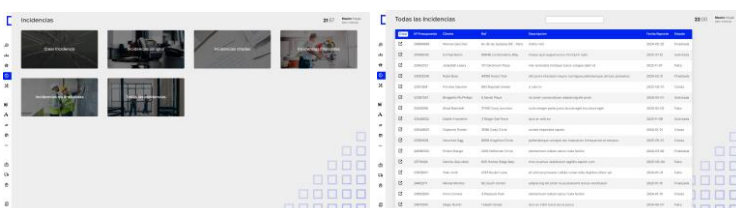
- Desde el menú lateral, selecciona "Incidencias".

Añadir una Nueva Incidencia:

- Haz clic en "Crear Incidencia".
- Describe el problema y añade cualquier información relevante.
- Haz clic en "Guardar" para registrar la incidencia.

Modificar o Cerrar Incidencias Existentes:

- Selecciona una incidencia de la lista y actualiza su estado o detalles según sea necesario.



## Montajes

Accede a la Sección de Montajes:

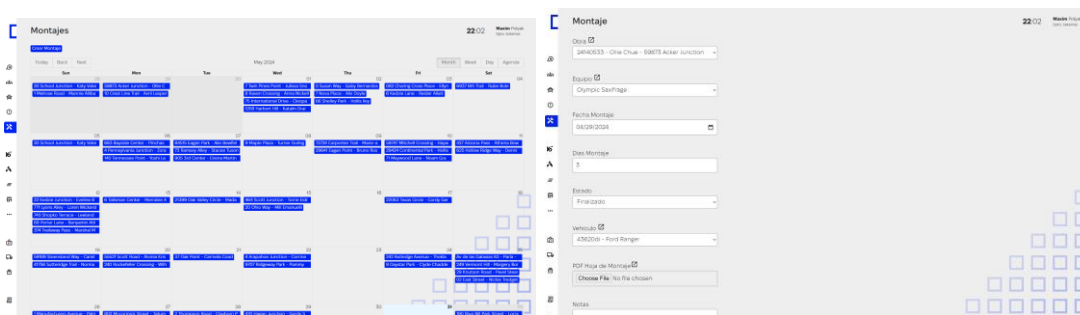
- Desde el menú lateral, selecciona "Montajes".

Programa un Nuevo Montaje:

- Haz clic en "Crear Montaje".
- Asigna un equipo y completa los detalles necesarios.
- Haz clic en "Guardar" para programar el montaje.

Visualiza y Modifica los Montajes Programados:

- Selecciona un montaje del calendario y actualiza los detalles según sea necesario.



## Producción

Accede a la Sección de Producción:

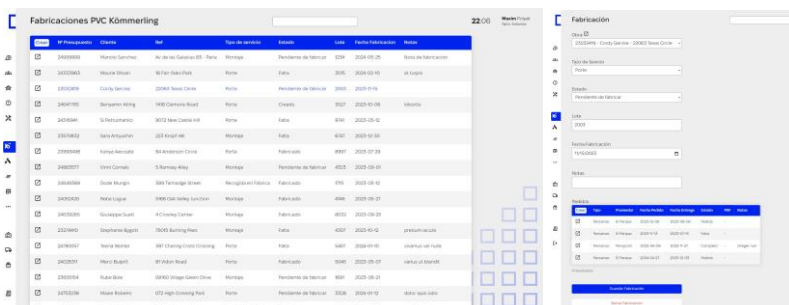
- Desde el menú lateral, selecciona "Producción".

Añadir o Modificar Registros de Fabricación:

- Haz clic en "Crear" para añadir una nueva entrada.
- Completa los detalles y guarda.

Administrar Pedidos de Materiales:

- Añade o modifica pedidos de materiales en los detalles de la fabricación.



## 7. DEMOSTRACIÓN

El despliegue de la aplicación ERP diseñada para Ventanas Vamin se realiza mediante dos instancias separadas en Amazon EC2 con Ubuntu 22, una para el front-end y otra para el back-end. A continuación, se detallan los pasos y procesos utilizados para llevar a cabo el despliegue de ambas partes de la aplicación.

### 7.2. Despliegue del Front-End en React

**Preparación del Entorno:** Se inicia configurando la instancia EC2 de Amazon con Ubuntu 22, asegurándose de que todas las actualizaciones necesarias estén instaladas. Además, se instala Nginx, Node.js y NPM, que son esenciales para la gestión de dependencias y la ejecución de la aplicación React.

**Clonación del Repositorio:** A través de SSH, se accede a la instancia y se clona el repositorio que contiene el código del front-end utilizando Git.

**Instalación de Dependencias:** Dentro del directorio del proyecto, se ejecuta `npm install` para instalar todas las dependencias necesarias definidas en el archivo `package.json`.

**Compilación y Despliegue:** Se realiza la compilación del proyecto con `npm run build`, que genera una versión optimizada del proyecto. Luego, se configura un servidor web Nginx para servir los archivos estáticos resultantes.

**Configuración de Seguridad:** Se establecen reglas de seguridad adecuadas en el grupo de seguridad de la instancia EC2 para permitir el tráfico al puerto donde se sirve la aplicación.

### 7.3. Despliegue del Back-End en PHP

**Configuración de la Instancia:** Al igual que con el front-end, se prepara otra instancia EC2 con Ubuntu 22, asegurando que el sistema esté actualizado y configurando el servidor web Nginx junto con PHP.

**Clonación del Repositorio del Back-End:** Se clona el repositorio del back-end en la instancia, posicionándose en el directorio adecuado para comenzar la configuración.

**Instalación de Dependencias:** se utiliza Composer para gestionar e instalar dependencias.

**Configuración de la Base de Datos:** Se configura MySQL, que es utilizado por el back-end para almacenar los datos de la aplicación. Se asegura la conexión entre PHP y MySQL mediante la configuración adecuada en los archivos de entorno del proyecto.

**Puesta en Marcha:** Finalmente, se configura el archivo de servidor de Nginx para dirigir las solicitudes al index.php del proyecto.

#### 7.4. Sistema de Despliegue Continuo

Para automatizar y facilitar los procesos de actualización de la aplicación, se implementa un sistema de despliegue continuo utilizando GitHub Actions:

**Configuración de GitHub Actions:** Se crea un workflow en GitHub Actions para cada repositorio que se dispara con cada push a la rama principal de los repositorios.

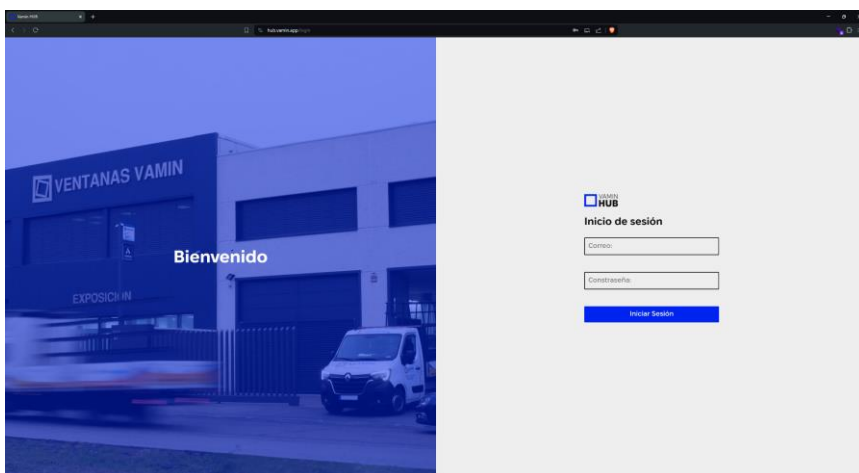
**Conexión SSH y Actualización del Código:** El workflow incluye pasos para conectarse vía SSH a las instancias EC2. Una vez conectado, se ejecutan comandos para hacer pull de los últimos cambios del repositorio correspondiente.

**Ejecución de Scripts de Despliegue:** Tras actualizar el código, se ejecutan scripts como por ejemplo para la compilación del código para React.

**Reinicio de Servicios:** Finalmente, se reinician los servicios necesarios, como Nginx o PHP-FPM, para asegurar que los nuevos cambios sean cargados y la aplicación esté actualizada.

Este sistema de despliegue continuo no solo optimiza el tiempo y esfuerzo necesarios para actualizar la aplicación, sino que también reduce el margen de error humano y aumenta la confiabilidad del proceso de despliegue.

(Junto al código de cada una de las dos instancias se encuentra un archivo README.md en el cual se puede encontrar el paso a paso más detallado para una instalación completa, también se puede usar en <https://hub.vamin.app/>)





## 8. BIBLIOGRAFÍA

PHP. (2024). PHP: Documentation. PHP. Último acceso 31 de mayo de 2024.  
<https://www.php.net/docs.php>

Laravel. (2024). Laravel. GitHub. Último acceso 31 de mayo de 2024. <https://github.com/laravel>

TanStack. (2024). TanStack Table: Headless UI for Building Tables. TanStack. Último acceso 31 de mayo de 2024. <https://tanstack.com/table/latest>

iamkun. (2024). Day.js: Fast 2kB JavaScript Date Library. Day.js. Último acceso 31 de mayo de 2024. <https://day.js.org/>

jquense. (2024). react-big-calendar. npm. Último acceso 31 de mayo de 2024.  
<https://www.npmjs.com/package/react-big-calendar>

remix-run. (2024). react-router-dom. npm. Último acceso 31 de mayo de 2024.  
<https://www.npmjs.com/package/react-router-dom>

Netlify. (2024). Vite PWA. Netlify. Último acceso 31 de mayo de 2024.  
<https://vite-pwa-org.netlify.app/>

pd4d10. (2024). vite-plugin-svgr. npm. Último acceso 31 de mayo de 2024.  
<https://www.npmjs.com/package/vite-plugin-svgr>

Ihechikara Vincent Abba (2022). What is an ORM? The Meaning of Object-Relational Mapping Database Tools. freeCodeCamp. Último acceso 31 de mayo de 2024.  
<https://www.freecodecamp.org/news/what-is-an-orm-the-meaning-of-object-relational-mapping-database-tools/>

Fin del documento